Part 1 – Nitin (Team leader): Repository, shared training, data efficiency, website server

This section explains every function and every block of code that Nitin wrote or owns. Read it with the files open. Each code block is copied from the real file. Line numbers come from the file as it is in the repository (folder `Backend/`).

What my part is

Nitin is the team leader. His part is the glue that makes the whole project fair and repeatable. He wrote `src/utils.py`, which gives everyone the same seed method, the same device choice and the same folder paths. He wrote `src/train.py`, which holds one shared `train()` function. Every model in the project (our own CNN, ResNet-18, MobileNetV2) is trained and scored by that one function, so the numbers can be compared. He ran the data-efficiency study in `notebooks/05_data_efficiency.ipynb`, which trains the two main models on 5% to 100% of the training images. He wrote `check_setup.py`, which checks each laptop before the team starts. He owns `results/metrics.csv`, the table where every finished run is recorded, and the root `README.md`. The website server `app/server.py` and the Grad-CAM control experiment `src/gradcam_sanity_check.py` were also added to his part, and `.gitignore` and `requirements.txt` keep the repository clean and the packages the same on every laptop.

Definitions

Simple meanings of the technical words used in the code and in this section.

Term	Plain meaning (used in our code)
Seed	A starting number for a random-number generator. The same seed gives the same "random" numbers, so a run can be repeated.
Random-number generator	A tool that produces numbers that look random. Python, numpy and PyTorch each have their own.
Reproducibility	Getting the same result again when you run the same code with the same seed on the same kind of machine.
Epoch	One full pass over all the training images.
Batch	A small group of images (64 here) that is used for one weight update.
Iteration / step	One batch: forward pass, loss, backward pass, weight update.
Weights (parameters)	The numbers inside the network that are learned during training.
Forward pass	Running an image through the network to get the scores.
Scores (logits)	The raw numbers the network outputs for each class before turning them into probabilities.
Loss	A number that says how wrong the model is. Training tries to make it smaller.
Cross-entropy loss	The standard loss for choosing one class out of several. `torch.nn.CrossEntropyLoss` in our code.
Gradient	How much the loss would change if one weight changed a little. It tells the optimiser which way to move each weight.
Backward pass (`backward`)	Work out the gradient of the loss for every weight.
Optimiser	The rule that changes the weights using the gradients.

Term	Plain meaning (used in our code)
Adam	A popular optimiser. It adapts the step size for each weight. Our optimiser.
Learning rate	The size of each weight update. Bigger means bigger steps.
`zero_grad`	Clears the gradients from the last batch, so they do not add up across batches.
`step`	Applies the optimiser's update to the weights.
Training mode / `model.train()`	Switches layers such as BatchNorm and Dropout to the behaviour used during training.
Evaluation mode / `model.eval()`	Switches the same layers to the behaviour used for scoring. BatchNorm uses saved averages, Dropout is off.
`torch.no_grad()`	A block where PyTorch does not track gradients. Used for scoring, which saves memory and time.
`requires_grad`	A flag on each weight. If it is False, the weight is never updated. Frozen layers have it set to False.
Convolution (Conv2d)	A small filter that slides over the image and finds local patterns such as edges and textures.
BatchNorm	A layer that rescales the values of each feature map using the batch's average and spread. It helps training to be stable.
ReLU	An activation function. It keeps positive values and turns negative values into zero.
MaxPool	Keeps the largest value in each 2 x 2 square, which halves the width and height.
Linear layer	A fully connected layer. It turns a list of features into one score per class.
Dropout	Randomly switches off some values during training to reduce over-fitting. Our scratch CNN has it off.
Over-fitting	When a model learns the training images too well and does worse on new images.
Under-training	When a model has not had enough weight updates to learn well.
Validation set	The 15% of the images we use to make decisions during training (here: which epoch to keep).
Test set	The 15% of the images we score only once, at the end, for the final number.
Split	The division of all images into train, validation and test. Fixed by `data/splits.csv`.
Stratified	Each class keeps the same share in every part of a split or subset.
Fraction	The share of the training images that a run uses (0.05 means 5%).
Best-epoch checkpoint	A saved copy of the weights from the epoch with the best validation accuracy.
`state_dict`	A Python dictionary that holds all the weights (and BatchNorm averages) of a model, by name.
`load_state_dict`	Puts saved weights back into a model.
`torch.save` / `torch.load`	Write and read a file with tensors or a state dictionary. `weights_only=True` loads only plain numbers, which is safer.
`map_location="cpu"`	Loads saved weights onto the CPU, even when they were saved from a GPU.
Tensor	PyTorch's name for a multi-dimensional array of numbers.
Device	The chip that does the work: `"cuda"` (NVIDIA GPU) or `"cpu"`.
CUDA	NVIDIA's system that lets PyTorch run on the GPU.
cuDNN	NVIDIA's library of fast convolution routines. Two switches control its behaviour in `set_seed`.
`cudnn.deterministic`	When True, the GPU always uses the same convolution method, so results repeat.
`cudnn.benchmark`	When True, the GPU tests several methods and keeps the fastest. We switch it off for repeatability.
Pixel values (bytes, 0 to 255)	The raw colour values of an image, stored as whole numbers.

Term	Plain meaning (used in our code)
`uint8`	A data type for whole numbers from 0 to 255. Our images are kept in this type to save memory.
In-memory cache	Data read once and kept in RAM or GPU memory so it is not read again.
`permute` / `contiguous`	Reorder the axes of a tensor (here from height-width-colour to colour-height-width), and make the memory layout continuous.
Normalisation	Subtracting the mean and dividing by the standard deviation of each colour channel.
ImageNet mean and std	The average and spread of the colours in the ImageNet photos. The team uses these numbers for every model.
Pretrained model	A model whose weights were already learned on a big dataset (ImageNet here).
Transfer learning	Re-using a pretrained model for a new task.
Fine-tuning (full)	Training all layers of a pretrained model, usually with a small learning rate.
Frozen backbone	The pretrained layers are kept fixed and only the new last layer learns.
Augmentation	Random changes to training images (here: 90-degree turns and flips) that make the model more robust.
Label-safe augmentation	Changes that do not change the true class. A rotated field is still a field.
Accuracy	The share of images whose predicted class is correct.
Macro-F1	The F1 score (a mix of precision and recall) calculated for each class and then averaged with equal weight for each class.
Precision	Of the images we labelled as a class, how many really were that class.
Recall	Of the images that really are a class, how many we found.
Confusion matrix	A table: one row per true class, one column per predicted class.
Mean	The average of several runs.
Standard deviation (SD)	How much the runs spread around the mean.
Sample SD, `ddof=1`	The SD that divides by (number of runs minus 1). Correct for a small sample of seeds. `ddof` is the "delta degrees of freedom" setting.
Mean ± SD	We write the average and the spread together, for example 93.69 ± 0.20.
Error bar	A line on a plot that shows the spread (here one SD).
Log scale	An axis where equal steps mean equal ratios (1, 10, 100, 1000), not equal differences.
`redirect_stdout`	A way to hide printed text inside a block. We use it to hide the per-epoch lines.
Resumable loop	A loop that skips work already saved, so it can be stopped and started again.
Idempotent	Running the same action twice gives the same final result, with no duplicates.
`__main__` guard	`if __name__ == "__main__":`. The code under it runs only when the file is started directly, not when it is imported.
`assert`	A check that stops the program with a message if the statement is False.
Docstring	The text in triple quotes under a `def` line that explains what the function does.
`pathlib.Path`	A tool to handle file paths as objects. `/` joins path parts.
`__file__` / `.resolve()`	The address of the current file, and its full, absolute form.
Constant	A name in capitals (for example `BATCH_SIZE`) that is not changed while the program runs.
Dictionary	A table of key and value pairs, for example model name to settings.
Default argument	A value a function uses when the caller does not give one.
Cache key	The name under which a cached item is stored in a dictionary.
`torch.Generator`	A separate random-number generator object, so one part of the code can have its own repeatable sequence.

Term	Plain meaning (used in our code)
`randperm`	Returns a random ordering of the numbers 0 to n minus 1. Used to shuffle the training rows.
`argmax`	The position of the largest value. We use it to pick the predicted class.
Softmax	Turns scores into probabilities that add up to 1. Used in the website server.
Top-k	The k largest values and their positions. Used for the top-3 guesses and for the Grad-CAM patches.
Grad-CAM	A method that colours the parts of an image that pushed the model to its answer.
Hook (PyTorch)	A function that PyTorch calls when a layer runs forward or backward. Grad-CAM uses hooks to read activations and gradients.
Feature map	The output of one filter over the image.
Hyperparameter	A setting chosen by us, not learned (batch size, epochs, learning rate).
FastAPI	A Python web framework. A decorated function such as `@app.get(...)` becomes a web address (route).
Route / endpoint	A web address and the function that answers it. For example `/api/models`.
JSON	A text format for data sent between the server and the web page.
Multipart upload	A way for a web form to send a file.
`UploadFile`, `File(...)`, `Form(None)`	FastAPI types. `File(...)` means the file is required. `Form(None)` means an optional text field.
Uvicorn	The program that runs a FastAPI app and listens for web requests.
CORS	Rules that let a web page on one address call the server on another address. We allow all, as the API document asks.
Static files	Files (HTML, CSS, JavaScript, images) that the server sends without running code.
Data URL (base64)	A way to put a whole image inside a text string, here the Grad-CAM picture.
RGBA / alpha	Colour with a fourth number for transparency.
Colour map (`jet`)	A table that turns a number from 0 to 1 into a colour from blue (low) to red (high).
Bilinear resize	Smooth resizing of an image by mixing neighbouring pixels.
Pillow (PIL)	The Python library that opens, converts and saves picture files.
Lock (`threading.Lock`)	A rule that lets only one thread run a block of code at a time.
Decompression bomb	A very large image file that is designed to use too much memory. Pillow raises an error for it.

---

4. The code, block by block

4.1 `src/utils.py` (all of it)

Why this file exists: three things are needed by every part. These are the same random seed method, the same way to choose a GPU or CPU, and the same folder names. If each person wrote their own, the results would not be comparable.

```
import random
from pathlib import Path

import numpy as np
import torch

# ---------- paths ----------
# This file is src/utils.py. .parent is the src folder, .parent.parent is the repo folder.
# Everyone imports these names, so nobody types a path by hand (our folder name has spaces).
ROOT = Path(__file__).resolve().parent.parent
```

```
DATA_DIR = ROOT / "data"
MODELS_DIR = ROOT / "models"                # saved weights (*.pth), not in git
RESULTS_DIR = ROOT / "results"
FIGURES_DIR = RESULTS_DIR / "figures"
SPLITS_CSV = DATA_DIR / "splits.csv"        # the fixed 70/15/15 split (part A)
METRICS_CSV = RESULTS_DIR / "metrics.csv"   # one row per finished run (train() in part D)


def set_seed(seed):
    """Make a run repeatable: the same seed gives the same random numbers."""
    random.seed(seed)                           # Python's own random numbers
    np.random.seed(seed)                        # numpy's random numbers
    torch.manual_seed(seed)                     # PyTorch's random numbers (CPU and GPU)
    torch.backends.cudnn.deterministic = True  # the GPU uses the same convolution method every time
    torch.backends.cudnn.benchmark = False     # do not let the GPU try methods and pick by speed
    # note: two different laptops can still give slightly different numbers


def get_device():
    """Return the GPU if PyTorch can use one, otherwise the CPU."""
    return torch.device("cuda" if torch.cuda.is_available() else "cpu")


if __name__ == "__main__":
    # Tiny self-check. Run it with:  python -m src.utils
    set_seed(0)
    a = torch.rand(3)
    set_seed(0)
    b = torch.rand(3)
    set_seed(1)
    c = torch.rand(3)
    assert torch.equal(a, b), "the same seed must give the same numbers"
    assert not torch.equal(a, c), "a different seed must give different numbers"
    for folder in (DATA_DIR, MODELS_DIR, RESULTS_DIR, FIGURES_DIR):
        assert folder.is_dir(), f"missing folder: {folder}"
    print("set_seed: OK (same seed gives the same numbers)")
    print("get_device:", get_device())
    print("paths: OK, repo folder is", ROOT)
```

Imports and paths (lines 14 to 29)

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
14	Loads Python's built-in random-number tool.	Standard library	`set_seed` must seed this one too, because some code picks random items with it.
15	Loads `Path`, a tool that treats a file path as an object.	`pathlib`	`/` joins path parts safely on Windows, so there are no long strings with backslashes. `os.path.join` also works but is older and clumsier.
17	Loads numpy, the number-array library.	numpy	scikit-learn (the split, the F1 score) and pandas use numpy's random numbers, so numpy is seeded too.
18	Loads PyTorch.	PyTorch	Our deep-learning library. The seed and the GPU check both come from it.
23	`ROOT` is the repository folder. `__file__` is this file's address. `.resolve()` makes it full. Each `.parent` goes up one folder.	Path relative to the file	Works on any laptop and in any folder. A typed `C:\Users\...` path would work on one laptop only.
24 to 27	`DATA_DIR`, `MODELS_DIR`, `RESULTS_DIR`, `FIGURES_DIR`: the four folders.	Constants	One place to change a folder name, and nobody types a path by hand.
28	`SPLITS_CSV`: the file with the fixed 70/15/15 split.	Constant	Written by Srivalli's `make_split`. Everybody reads the split from here.
29	`METRICS_CSV`: the results table.	Constant	`train()` adds one row per finished run. The brief asks for all results in `results/metrics.csv`.

`set_seed` (lines 32 to 39)

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
32	Defines the function. `seed` is any whole number.	Function, seed	The project uses seeds 0 to 4. The split uses 42.
33	The docstring says what the function is for.	Docstring	Anyone can read the purpose without reading the code.
34	Seeds Python's random numbers.	Random seed	Python's `random` picks some items in code.
35	Seeds numpy's random numbers.	Random seed	scikit-learn and pandas use numpy's generator.
36	Seeds PyTorch on CPU and GPU.	Random seed	PyTorch uses random numbers for starting weights, the order of batches, dropout and augmentation. This is the most important of the three.
37	Asks cuDNN to use the same convolution method every time.	`cudnn.deterministic`	Some fast methods give tiny different answers each run. This can be a little slower. We could skip it and accept small differences.
38	Stops cuDNN from timing methods and choosing the fastest.	`cudnn.benchmark`	The fastest method can change from run to run, which would break repeatability.
39	A note: two laptops can still give slightly different numbers.	Reproducibility limit	Different GPUs and driver versions round decimals differently. The report says so and reports mean and spread over several seeds.

`get_device` (lines 42 to 44)

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
42 to 43	Defines the function and its docstring.	Function	Every part calls `get_device()` instead of deciding by itself.
44	Asks PyTorch "can you use an NVIDIA GPU?". Returns `"cuda"` if yes, otherwise `"cpu"`.	Device	Later code calls `.to(device)`, so the same code runs on a GPU laptop and on a CPU laptop.

Self-check (lines 47 to 61)

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
47	Runs the lines below only when the file is started directly.	`__main__` guard	When another file imports `set_seed`, this block does not run.
49 to 54	Makes three random tensors: seed 0, seed 0 again, seed 1.	Test of a seed	The first two must match and the third must differ.
55 and 56	`assert` stops the program with a message if a statement is false.	Assertion	If someone breaks `set_seed`, running the file shows it at once. A full test framework would be more than we need.
57 and 58	Checks that the four folders exist.	Assertion	Catches a wrong path early.
59 to 61	Prints the result, the device and the repository folder.	Output	Any teammate can see at a glance that it worked.

4.2 `src/train.py` (the shared `train()`)

Why this file exists: the brief asks for one reusable training function, so every experiment (our own CNN, the transfer models, the data-efficiency study) is trained and recorded in the same way. The function is based on Tejash's notebook helper, because his settings already matched the team settings. Nitin added the `fraction` option and made it faster by keeping all images in memory.

4.2.1 Module docstring, imports and constants (lines 1 to 35)

```
import tempfile
import time
from pathlib import Path

import numpy as np
import pandas as pd
import torch
from PIL import Image
from sklearn.metrics import f1_score

from src import models
from src.dataset import IMAGENET_MEAN, IMAGENET_STD, get_loader
from src.utils import DATA_DIR, METRICS_CSV, MODELS_DIR, SPLITS_CSV, get_device, set_seed

BATCH_SIZE = 64
MAX_EPOCHS = 20
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1 to 17	The docstring: one shared function, team settings, the speed trick and the self-check command.	Docstring	The file explains itself to a teammate.
20 to 22	`tempfile` makes a throw-away folder (used only by the self-check). `time` is a stopwatch. `Path` handles file paths.	Standard library	The self-check must never touch the real results file.
24 to 28	numpy, pandas (tables), PyTorch, `Image` (opens picture files), `f1_score` (macro-F1).	Libraries	`f1_score` comes from scikit-learn, so we do not write our own.
30	`from src import models` loads Tejash's and Rachana's model file.	Import	`train()` builds models by name and does not copy their code.
31	Takes the ImageNet mean and std and `get_loader` from Srivalli's `dataset.py`.	Reuse	The same normalisation numbers for all models. `get_loader` picks exactly the same images as her code.
32	Takes the folders, the split file, `get_device` and `set_seed` from `utils.py`.	Reuse	One place for paths and seeds.
34	`BATCH_SIZE = 64`.	Hyperparameter	The team setting in `docs/plan.md`. A batch is the group of images used for one update.
35	`MAX_EPOCHS = 20`.	Hyperparameter	The team setting. One epoch is one pass over the training images.

4.2.2 Model names and `build_model` (lines 37 to 62)

```
# Model names. The scratch CNN names are the ones Tejash used in his runs.
SCRATCH = {
    "scratch_cnn": {"use_batchnorm": True, "depth": 3},                # the baseline
    "scratch_cnn_no_batchnorm": {"use_batchnorm": False, "depth": 3},  # ablation 1
    "scratch_cnn_depth2": {"use_batchnorm": True, "depth": 2},         # ablation 2
}
# Transfer models (Rachana, part C): name -> (backbone, freeze the backbone?)
TRANSFER = {
    "resnet18_frozen": ("resnet18", True),
    "resnet18_full": ("resnet18", False),
    "mobilenet_v2_frozen": ("mobilenet_v2", True),
    "mobilenet_v2_full": ("mobilenet_v2", False),
}


def build_model(model_name):
    """Return (model, learning rate) for one of our model names."""
    if model_name in SCRATCH:
        return models.build_scratch_cnn(**SCRATCH[model_name]), 1e-3
    if model_name in TRANSFER:
        if not hasattr(models, "build_transfer_model"):
            raise NotImplementedError("build_transfer_model is not in src/models.py yet (Rachana, part C)")
        backbone, freeze = TRANSFER[model_name]
        # Team default: 0.001 when only the new last layer learns, 0.0001 when everything is fine-tuned.
```

```
        return models.build_transfer_model(backbone, freeze), (1e-3 if freeze else 1e-4)
    raise ValueError(f"unknown model name {model_name!r}; use one of {sorted([*SCRATCH, *TRANSFER])}")
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
37 to 42	`SCRATCH` links three names to the switches of `build_scratch_cnn`: BatchNorm on or off, and 3 or 2 blocks.	Dictionary, default settings	These are the names Tejash used, so his rows and ours match in `metrics.csv`.
43 to 49	`TRANSFER` links four names to (backbone, frozen or not).	Dictionary, frozen vs full	Frozen means only the new last layer learns. Full means everything learns.
52	`build_model(model_name)` returns the model and the learning rate.	Function, return value	The learning rate is chosen here so every caller gets the right pair.
54 and 55	For a scratch name: builds the scratch CNN with the switches from `SCRATCH`, learning rate 0.001.	Learning rate	0.001 is a usual starting rate for a network trained from random weights.
56 to 58	For a transfer name: checks that `build_transfer_model` exists. If not, stops with a clear message.	Guard, clear error	While Rachana's code was not yet in place this gave a readable message instead of a crash.
59 to 61	Looks up the backbone and the freeze switch, then returns the model and `1e-3` (frozen) or `1e-4` (full).	Fine-tuning learning rate	A lower rate for full fine-tuning avoids damaging the pretrained weights. We could use one rate for everything, but that is a worse choice for pretrained models.
62	For any other name: stops with an error that lists the valid names.	Input check	A typo fails at once with a helpful message.

4.2.3 Fast data: all images kept in memory (lines 65 to 80)

```
# ---------- fast data: all images in memory ----------
_cache = {}  # filled by the first train() call and reused by every later call in the same Python session


def _load_all_images(device):
    """Read all 27,000 images once. Returns (images, labels, row_of_path) with everything on the device."""
    key = str(device)
    if key not in _cache:
        df = pd.read_csv(SPLITS_CSV)
        print(f"Reading {len(df)} images into memory (one time, about 20 seconds)...")
        pictures = [np.array(Image.open(DATA_DIR / path).convert("RGB")) for path in df["path"]]
        images = torch.from_numpy(np.stack(pictures)).permute(0, 3, 1, 2).contiguous()  # (N, 3, 64, 64) bytes
        labels = torch.tensor(df["label"].values)
        row_of_path = {path: row for row, path in enumerate(df["path"])}
        _cache[key] = (images.to(device), labels.to(device), row_of_path)
    return _cache[key]
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
66	`_cache` is a dictionary that remembers the loaded data.	Cache	The first `train()` call fills it. Later calls in the same session reuse it.
71	The key is the device name, for example `"cuda"`.	Cache key	Data is stored once per device.
72 to 74	If not cached: reads `splits.csv` and prints a message about the time.	Lazy loading	The first call is slow, the rest are fast.
75	Opens every picture, converts it to RGB and makes a numpy array of 64 x 64 x 3 bytes.	Pixel values 0 to 255, `uint8`	Reading and decoding JPEGs in every epoch was slow (around 12 seconds per epoch just to read, from the log). Kept as bytes, all images take about 330 MB. A DataLoader with workers would be the other option, but it is fragile on Windows.

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
76	Stacks them into one block, moves the colour axis to the second place, and makes it contiguous. Shape is (27000, 3, 64, 64).	`permute`, `contiguous`, tensor	PyTorch expects colour channels before height and width.
77	Puts the labels into a tensor.	Tensor	Labels are read by row, like the images.
78	Builds a lookup from file path to row number.	Dictionary lookup	The split files give paths, and this turns each path into a row number in the big block.
79	Moves images and labels to the device (GPU here) and stores them with the lookup.	`.to(device)`	Batches are then picked on the GPU without copying from disk.
80	Returns the cached tuple.	Return value	Every caller gets the same data.

4.2.4 Choosing the rows of each split (lines 83 to 87)

```
def _rows(split, fraction, seed, row_of_path, device):
    """Row numbers of one split. For train, a stratified share of it, chosen by get_loader's own rule."""
    # get_loader only reads splits.csv here (no images), so we reuse its exact subset rule.
    frame = get_loader(split, fraction=fraction if split == "train" else 1.0, seed=seed).dataset.df
    return torch.tensor([row_of_path[path] for path in frame["path"]], device=device)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
83	`_rows` returns the row numbers for one split.	Function	Used three times: train, validation and test.
86	Asks `get_loader` for the split. For `train` it passes `fraction`. For `val` and `test` it passes 1.0, so they never shrink. Takes the `.dataset.df` table.	Stratified subset, reuse	No images are read here, only the table. We reuse Srivalli's exact rule, so the 5% subset is the same images her code would pick.
87	Turns the paths into row numbers and moves them to the device.	Lookup	Row numbers index the big image block.

4.2.5 Bytes to numbers, and the augmentation (lines 90 to 106)

```
def _prepare(batch_bytes, augment):
    """Bytes (0 to 255) to normalised floats. With augment: random 90-degree turns and flips, per image."""
    x = batch_bytes.float() / 255.0
    if augment:
        n = x.shape[0]
        turns = torch.randint(0, 4, (n,), device=x.device)
        for k in (1, 2, 3):
            chosen = turns == k
            if chosen.any():
                x[chosen] = torch.rot90(x[chosen], k, dims=(2, 3))
        flip_sideways = torch.rand(n, device=x.device) < 0.5
        flip_upside_down = torch.rand(n, device=x.device) < 0.5
        x[flip_sideways] = x[flip_sideways].flip(3)
        x[flip_upside_down] = x[flip_upside_down].flip(2)
    mean = torch.tensor(IMAGENET_MEAN, device=x.device).view(1, 3, 1, 1)
    std = torch.tensor(IMAGENET_STD, device=x.device).view(1, 3, 1, 1)
    return (x - mean) / std
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
90	Defines `_prepare`. It takes a batch of bytes and a yes/no for augmentation.	Function	Training uses `augment=True`. Scoring uses `augment=False`.
92	Converts to decimals and divides by 255, so values are from 0 to 1.	Scaling pixels	The same first step as `ToTensor` in `dataset.py`.
93	Starts the augmentation block only when asked.	Augmentation only in training	Validation and test images must stay as they are.

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
94 and 95	Gives each image a random number 0, 1, 2 or 3, meaning 0, 90, 180 or 270 degrees.	Random choice per image	Uses PyTorch's random numbers, which the seed controls.
96 to 99	For turns 1, 2 and 3, rotates the chosen images in their height-width plane. Skips a turn if nobody picked it.	`torch.rot90`	Doing the work on the whole batch with masks is fast on the GPU. The rotation is exact, so no pixels are blurred.
100 and 101	Makes two random yes/no masks, one for a sideways flip and one for an upside-down flip, at 50% each.	Masks, random flips	Each image gets its own random choice.
102 and 103	Flips the chosen images. `flip(3)` flips left-right (width), `flip(2)` flips top-bottom (height).	Flip	A satellite has no "up", so a flipped field is still a field. These are the same changes as `augment()` in `dataset.py`.
104 and 105	Makes the ImageNet mean and std as tensors with shape (1, 3, 1, 1).	Broadcasting	Each colour channel gets its own mean and std.
106	Subtracts the mean and divides by the std.	Normalisation	The team uses the same numbers for all models, so the comparison is fair.

Why the flip is written with masks: the code flips only the chosen images with `x[mask] = x[mask].flip(3)`. This is correct and simple. A possible alternative is to flip each image in a loop, which is slower.

4.2.6 Scoring without training (lines 109 to 122)

```
def _score(model, rows, images, labels, loss_function):
    """Average loss, accuracy and predictions for some rows, without augmentation."""
    model.eval()
    total_loss = 0.0
    predictions = []
    with torch.no_grad():
        for start in range(0, len(rows), 256):
            batch = rows[start:start + 256]
            scores = model(_prepare(images[batch], augment=False))
            total_loss += loss_function(scores, labels[batch]).item() * len(batch)
            predictions.append(scores.argmax(dim=1))
    predictions = torch.cat(predictions)
    accuracy = (predictions == labels[rows]).float().mean().item()
    return total_loss / len(rows), accuracy, predictions
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
109	Defines `_score`. It is used for validation and for the test set.	Function	One scoring code for both splits.
111	Puts the model in evaluation mode.	`model.eval()`	BatchNorm uses its saved averages instead of the batch's, and dropout is off. Without this, scores would change from call to call.
112 and 113	Starts a running loss total and an empty list of predictions.	Accumulators	
114	Turns off gradient tracking for the block.	`torch.no_grad()`	Nothing is learned here, so gradients are not needed. This saves memory and time.
115 and 116	Goes through the rows in groups of 256.	Batches for scoring	256 is bigger than 64 because nothing is stored for learning. The answer is the same.
117	Runs the model on the normalised, non-augmented images.	Forward pass	
118	Adds the loss times the group size to the total.	Weighted sum	`loss_function` gives the mean over the group, so multiplying by the size makes a total.

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
119	Keeps the predicted class for each image (the highest score).	`argmax`	The predicted class is the position of the largest score.
121	Accuracy is the share of predictions that equal the true labels.	Accuracy	It is a fraction from 0 to 1 here.
122	Returns the average loss, the accuracy and the predictions.	Return value	The predictions are reused for macro-F1 on the test set.

4.2.7 Writing the results row (lines 125 to 136)

```
def _save_row(path, row):
    """Add one row to the results file. A rerun of the same (model, fraction, seed) replaces its old row."""
    columns = ["model", "fraction", "seed", "accuracy", "macro_f1"]
    path = Path(path)
    old = pd.read_csv(path) if path.exists() and path.stat().st_size > 0 else pd.DataFrame(columns=columns)
    if old.empty:
        new = pd.DataFrame([row], columns=columns)
    else:
        same_run = ((old["model"] == row["model"]) & np.isclose(old["fraction"].astype(float), row["fraction"])
                    & (old["seed"] == row["seed"]))
        new = pd.concat([old.loc[~same_run], pd.DataFrame([row], columns=columns)], ignore_index=True)
    new.to_csv(path, index=False, lineterminator="\n")
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
127	Lists the five columns the brief asks for.	Schema	The file keeps exactly these columns.
128	Turns the path into a `Path` object.	`pathlib`	
129	Reads the old file if it exists and is not empty. Otherwise starts an empty table.	Read or create	The first run creates the file.
130 and 131	If the old table is empty, the new table is just the new row.	Edge case	
132 to 134	Finds the old rows that match the same model, fraction and seed. `np.isclose` compares fractions safely (0.05 is not stored exactly in binary).	Idempotent write, floating-point comparison	A rerun of the same run must not create a duplicate.
135	Keeps all rows that do not match, and adds the new row.	Replace instead of duplicate	The file keeps one row per run. Other rows are never changed.
136	Writes the file with `\n` line endings.	`to_csv`	The same line endings on every laptop, so git sees no fake changes.

Note: `accuracy` and `macro_f1` are stored as decimals from 0 to 1 (for example 0.9827). Percentages are made later, in the notebook and in the server.

4.2.8 The `train()` function (lines 139 to 227)

```
def train(model_name, fraction, seed, epochs=MAX_EPOCHS, metrics_csv=METRICS_CSV, save_weights=None):
    """Train one model with the team settings and add ONE row to results/metrics.csv.

    model_name:   one of SCRATCH or TRANSFER above
    fraction:     share of the TRAIN split to use, from 0.05 to 1.0 (stratified). Validation and test never shrink.
    seed:         makes the run repeatable (starting weights, which images, their order, augmentation)
    save_weights: keep the best-epoch weights in models/ (not in git). Default: only when fraction is 1.0.

    The best epoch is chosen on the VALIDATION set. The test set is scored ONCE, after training.
    Returns a dict with the result and the training history.
```

```
    """
    if not 0 < fraction <= 1:
        raise ValueError("fraction must be above 0 and at most 1")
    device = get_device()
    set_seed(seed)

    images, labels, row_of_path = _load_all_images(device)
    train_rows = _rows("train", fraction, seed, row_of_path, device)
    val_rows = _rows("val", 1.0, seed, row_of_path, device)
    test_rows = _rows("test", 1.0, seed, row_of_path, device)

    model, learning_rate = build_model(model_name)
    model = model.to(device)
    loss_function = torch.nn.CrossEntropyLoss()
    # Only the layers that are allowed to learn go to the optimiser (matters for the frozen models).
    # foreach=False is the same setting Tejash used in his runs.
    optimizer = torch.optim.Adam([p for p in model.parameters() if p.requires_grad],
                                 lr=learning_rate, foreach=False)
    shuffler = torch.Generator().manual_seed(seed)  # decides the order of the images, so a run repeats

    history = {"train_loss": [], "train_accuracy": [], "val_loss": [], "val_accuracy": []}
    best_accuracy, best_epoch, best_weights = -1.0, 0, None
    started = time.perf_counter()

    for epoch in range(1, epochs + 1):
        epoch_started = time.perf_counter()
        model.train()
        total_loss = torch.zeros((), device=device)
        correct = torch.zeros((), device=device)
        seen = 0

        order = train_rows[torch.randperm(len(train_rows), generator=shuffler).to(device)]
        for start in range(0, len(order), BATCH_SIZE):
            batch = order[start:start + BATCH_SIZE]
            if len(batch) < 2:  # BatchNorm cannot work with a single image
                continue
            x = _prepare(images[batch], augment=True)
            y = labels[batch]
            optimizer.zero_grad()
            scores = model(x)
            loss = loss_function(scores, y)
            loss.backward()
            optimizer.step()
            total_loss += loss.detach() * len(batch)
            correct += (scores.argmax(dim=1) == y).sum()
            seen += len(batch)

        val_loss, val_accuracy, _ = _score(model, val_rows, images, labels, loss_function)
        history["train_loss"].append(total_loss.item() / seen)
        history["train_accuracy"].append(correct.item() / seen)
        history["val_loss"].append(val_loss)
        history["val_accuracy"].append(val_accuracy)
        print(f"{model_name} | fraction {fraction:g} | seed {seed} | epoch {epoch:02d}/{epochs} | "
              f"train loss {history['train_loss'][-1]:.4f}, acc {history['train_accuracy'][-1]:.3f} | "
              f"val loss {val_loss:.4f}, acc {val_accuracy:.3f} | {time.perf_counter() - epoch_started:.1f} s")

        if val_accuracy > best_accuracy:  # keep the best epoch, chosen on validation
            best_accuracy, best_epoch = val_accuracy, epoch
            best_weights = {name: value.detach().cpu().clone() for name, value in model.state_dict().items()}

    model.load_state_dict(best_weights)

    # The test set is used here, once, and nothing is decided from it.
    _, accuracy, predictions = _score(model, test_rows, images, labels, loss_function)
    macro_f1 = f1_score(labels[test_rows].cpu().numpy(), predictions.cpu().numpy(), average="macro")
    row = {"model": model_name, "fraction": fraction, "seed": seed, "accuracy": accuracy, "macro_f1": macro_f1}
    _save_row(metrics_csv, row)

    if save_weights is None:
        save_weights = fraction == 1.0
    if save_weights:
        MODELS_DIR.mkdir(exist_ok=True)
        torch.save(model.state_dict(), MODELS_DIR / f"{model_name}_frac{fraction:g}_seed{seed}.pth")

    seconds = time.perf_counter() - started
    print(f"TEST (best epoch {best_epoch}): accuracy {accuracy:.4f}, macro-F1 {macro_f1:.4f} "
          f"| {seconds / 60:.1f} min, {seconds / epochs:.1f} s per epoch")
    return {**row, "best_epoch": best_epoch, "val_accuracy": best_accuracy,
            "seconds_per_epoch": seconds / epochs, "history": history}
```

Part 4.2.8a: the function header, checks and setup (lines 139 to 171)

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
139	Defines `train`. Only three things are required: model name, fraction and seed.	Function with defaults	`epochs`, `metrics_csv` and `save_weights` have defaults, so a normal call is short. The default `metrics_csv` is the real results file.
150 and 151	Stops with an error if `fraction` is not above 0 and at most 1.	Input check	A wrong value fails at once, not after an hour of training.
152	Picks the device.	Device	GPU if available.
153	Sets the seed.	Seed	The seed is set before anything random happens, so the whole run repeats.
155	Loads all images into memory (first call only).	Cache	
156 to 158	Gets the row numbers of train (reduced by `fraction`), validation and test (full).	Fixed split	Validation and test never shrink. Only the training rows follow `fraction`.
160	Builds the model by name and gets its learning rate.	`build_model`	
161	Moves the model to the device.	`model.to(device)`	Same code on GPU and CPU.
162	Creates the loss function.	Cross-entropy	The standard loss for choosing one class out of several. Same as Tejash's.
165 and 166	Creates Adam. Only parameters with `requires_grad` True are given to it. Sets the learning rate. `foreach=False` is the same switch Tejash used.	Optimiser, `requires_grad`	Frozen layers must not change. Only the new last layer is updated in frozen models.
167	Makes a random generator from the seed.	`torch.Generator`	It decides the shuffle order, so the order repeats. It is separate from the global generator.
169	Creates empty lists for the history.	Bookkeeping	The history is returned, and notebooks can plot it. It is not written to disk by `train()`.
170	Sets the best accuracy to -1, the best epoch to 0 and no weights yet.	Best-epoch tracking	Any real accuracy is higher than -1, so the first epoch always becomes the first best.
171	Starts the stopwatch.	Timing	

Part 4.2.8b: the epoch loop, the training steps (lines 173 to 194)

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
173	Starts the loop over epochs, 1 to 20.	Epoch	One epoch is one full pass over the training rows.
174	Starts a timer for this epoch.	Timing	Used for the printed seconds.
175	Switches to training mode.	`model.train()`	BatchNorm uses batch statistics and dropout is active. Note: in frozen models BatchNorm still updates its averages.
176 and 177	Makes counters for the loss and the number of correct answers. They live on the GPU.	Counters	Reading them after every batch would slow training, so they are read once per epoch.
178	Counts images seen in this epoch.	Counter	Used to divide the totals.

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
180	Shuffles the training rows with the seeded generator and moves them to the device.	Shuffle	A different order each epoch helps learning. The seed makes the order repeatable.
181	Goes through the shuffled rows in groups of 64.	Batch	
182	Takes one group.	Batch slice	
183 and 184	Skips a group of only one image.	BatchNorm limit	BatchNorm cannot work with one image. With 18,900 images and batches of 64 the last batch has 18,900 mod 64 = 20 images (for 100%), so this line rarely matters, but it prevents a crash.
185	Prepares the images with augmentation.	Augmentation only in training	
186	Gets the true labels of the batch.	Labels	
187	Clears the old gradients.	`zero_grad`	Otherwise gradients add up across batches.
188	Runs the forward pass to get the scores.	Forward pass	
189	Computes the loss between the scores and the true labels.	Cross-entropy	
190	Works out the gradient of the loss for every weight.	`backward`	This is the learning step: it finds which way each weight should move.
191	Moves every weight one step with Adam.	`step`	Core of training.
192	Adds the loss times the group size to the epoch total (no gradient kept).	`detach`	`detach` removes the link to the gradient graph, so memory is not kept.
193	Counts the correct answers in the group.	Accuracy count	
194	Adds the group size to `seen`.	Counter	

Part 4.2.8c: scoring, the history and the best epoch (lines 196 to 207)

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
196	Scores the whole validation set without augmentation.	Validation	This is the practice exam.
199 to 200	Stores the average loss and accuracy for training and validation.	History	Curves for the report (learning curves) and for over-fitting checks.
201 to 203	Prints one line for the epoch, with losses, accuracies and seconds.	Progress	Makes it easy to watch a laptop run.
205	If this epoch's validation accuracy is higher than the best so far...	Best-epoch selection on validation	The choice uses validation only, never the test set.
206	...remembers the epoch number and the accuracy.	Best epoch	The best epoch is printed at the end and returned. It is not saved in the CSV.
207	Keeps a copy of all weights and BatchNorm averages on the CPU.	`state_dict`, `clone`	The copy is taken before the next epoch changes the model. Copying to the CPU frees GPU memory.

Part 4.2.8d: the test set, the results row and the saved weights (lines 209 to 221)

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
209	Puts the best epoch's weights back into the model.	`load_state_dict`	We report the best version, not the last one.
211	Comment: the test set is used here, once.	Test set	Nothing is decided from the test score.
212	Scores the test set with the best weights.	Test set	Returns accuracy and predictions.
213	Computes macro-F1 on the test set with `average="macro"`.	Macro-F1	Every class counts equally, so a weak small class is not hidden.
214	Makes the row: model, fraction, seed, accuracy, macro-F1.	Result row	
215	Writes the row to the results file, replacing an old row of the same run.	`_save_row`	One row per run.
217 and 218	If `save_weights` was not given, it is True only when `fraction` is exactly 1.0.	Default argument	Rohan's evaluation needs the full-data models. Smaller runs do not save weights, to save disk space.
219 to 221	Makes the `models/` folder if needed, then saves the state dictionary as `<model>_frac<fraction>_seed<seed>.pth`.	Checkpoint	For fraction 1.0 this gives names such as `mobilenet_v2_full_frac1_seed0.pth`. The folder is ignored by git.

Part 4.2.8e: the end of the function (lines 223 to 227)

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
223	Stops the stopwatch.	Timing	
224 and 225	Prints the test line with the best epoch, the accuracy, macro-F1, the minutes and the seconds per epoch.	Output	
226 and 227	Returns the row, the best epoch, the best validation accuracy, the time per epoch and the history.	Return value	Notebook 05 reads `accuracy` and `best_epoch` from this dictionary.

4.2.9 Self-check (lines 230 to 239)

```
if __name__ == "__main__":
    # Tiny check: 2 epochs on 5% of the data, written to a temporary file (the real metrics.csv is NOT touched).
    with tempfile.TemporaryDirectory() as folder:
        results_file = Path(folder) / "metrics.csv"
        train("scratch_cnn", fraction=0.05, seed=0, epochs=2, metrics_csv=results_file, save_weights=False)
        table = pd.read_csv(results_file)
        assert len(table) == 1 and 0 < table["accuracy"][0] <= 1, "one row with a sensible accuracy"
        train("scratch_cnn", fraction=0.05, seed=0, epochs=1, metrics_csv=results_file, save_weights=False)
        assert len(pd.read_csv(results_file)) == 1, "a rerun must replace its row, not add a duplicate"
    print("train: OK (one row written, a rerun replaced it)")
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
232	Makes a temporary folder that is deleted at the end.	`tempfile`	The real `metrics.csv` is never touched.
233 and 234	Trains the scratch CNN for 2 epochs on 5% with seed 0, into the temporary file, with no weight file.	`__main__` guard, test run	Run it with `python -m src.train`.
235 and 236	Reads the temporary file and checks it has one row with a sensible accuracy.	Assertion	
237 and 238	Runs again with the same run settings and checks there is still one row.	Idempotent write test	This tests the replace rule in `_save_row`.
239	Prints OK.	Output	

How it was tested (from the log, not re-run here): the self-check passed. A longer test, kept outside the repository, showed that the in-memory pixels equal the original loader's pixels, that every augmented image is one of the 8 turns or flips of the original, that the 5% subset is the same as Srivalli's `get_loader` picks, and that the train, validation and test rows do not overlap. Full 20-epoch runs gave a baseline mean of 92.79% over seeds 0 to 2 (Tejash's notebook: 92.72%). Speed was about 4.6 seconds per epoch on the RTX 3050.

Points to be careful about (honest notes):

- The docstring says the fraction is "from 0.05 to 1.0". The code accepts any value above 0 and up to 1. The smallest value used in the study is 0.05.
- BatchNorm statistics still update during training in the frozen transfer models, because `model.train()` is called on the whole model. The log lists this as a report caveat.
- The fraction is passed to `get_loader`, which uses scikit-learn's `train_test_split`. Each run builds its subset from the same seed, so the 5% subset of seed 0 is the same every time.

4.3 `src/models.py` (only the parts `train()` calls)

Tejash owns the scratch CNN and Rachana owns the transfer models. `train()` only uses two of their functions: it builds a model by name and, for transfer models, it freezes or unfreezes the backbone. Below are the parts it calls.

Signature of the scratch builder, called as `models.build_scratch_cnn(**SCRATCH[model_name])`:

```
def build_scratch_cnn(
    num_classes=10,
    use_batchnorm=True,
    depth=3,
    dropout=0.0,
):
    ...
    filter_counts = [32, 64, 128][:depth]
    layers = []
    input_channels = 3

    for output_channels in filter_counts:
        # Each block finds image features, then halves the image width and height.
        layers.append(nn.Conv2d(input_channels, output_channels, kernel_size=3, padding=1))
        if use_batchnorm:
            layers.append(nn.BatchNorm2d(output_channels))
        layers.append(nn.ReLU())
        layers.append(nn.MaxPool2d(kernel_size=2))
        input_channels = output_channels

    # Keep one value per feature map, whatever the final image size is.
    layers.append(nn.AdaptiveAvgPool2d(1))
    layers.append(nn.Flatten())
    if dropout > 0:
        layers.append(nn.Dropout(p=dropout))
    layers.append(nn.Linear(input_channels, num_classes))

    return nn.Sequential(*layers)
```

The transfer builder, called as `models.build_transfer_model(backbone, freeze)`. This is the freeze logic:

```
def build_transfer_model(name: str, freeze: bool = True):
    """Build a pretrained ImageNet model adapted for 10 EuroSAT classes.
    ...
    """
    if name == "resnet18":
        model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
    elif name == "mobilenet_v2":
        model = models.mobilenet_v2(weights=models.MobileNet_V2_Weights.DEFAULT)
    else:
        raise ValueError(f"Unsupported transfer model {name!r}; expected 'resnet18' or 'mobilenet_v2'")

    if freeze:
        # Freeze all pretrained weights first.
        for param in model.parameters():
            param.requires_grad = False
```

```
    # Replace the 1000-class ImageNet head with a new 10-class linear layer.
    # A new nn.Linear has requires_grad=True by default, so it always learns.
    if name == "resnet18":
        model.fc = nn.Linear(model.fc.in_features, 10)
    elif name == "mobilenet_v2":
        model.classifier[1] = nn.Linear(model.classifier[1].in_features, 10)

    return model
```

Lines (in `models.py`)	What it does (simple words)	Concept	Why it matters for `train()`
11 to 16 (signature)	The scratch builder takes the number of classes, the BatchNorm switch, the depth (2 or 3 blocks) and dropout.	Default arguments	`train()` passes only `use_batchnorm` and `depth` from its `SCRATCH` dictionary.
33 to 40	One loop per block: convolution, optional BatchNorm, ReLU, then MaxPool that halves the size.	Convolution block	Three blocks with 32, 64 and 128 filters is the baseline.
43 to 47	Average pooling, flatten, optional dropout, then a linear layer to 10 scores.	Head	The output is one score per class, which `CrossEntropyLoss` uses.
59 to 64	Loads the ImageNet-pretrained ResNet-18 or MobileNetV2 with torchvision.	Pretrained weights	The first run downloads them to the torch cache. Later runs use the cache.
66 to 69	If `freeze` is True, every parameter gets `requires_grad = False`.	Frozen backbone	`train()` gives the optimiser only the parameters that still have `requires_grad` True.
73 to 76	Replaces the 1000-class ImageNet output with a new 10-class linear layer.	Transfer head	The new layer is created with `requires_grad=True`, so it always learns, even when the backbone is frozen.

The learning-rate choice (1e-3 when frozen, 1e-4 when fully fine-tuned) is made in `build_model` in `train.py`, not here.

4.4 `notebooks/05_data_efficiency.ipynb` (all code cells)

Why this notebook exists: the brief asks for a data-efficiency study with several seeds at small data sizes. It compares our scratch CNN with the best transfer model on validation, MobileNetV2 fully fine-tuned. Every run goes through `train()`, so the numbers are comparable with Parts B and C. The text cells explain the question and the results. The code cells are below. The cell IDs are given because they are easier to find than cell numbers.

Setup cell (id 327b72df)

```
from pathlib import Path
import contextlib
import io
import os
import sys
import time

# Make imports work whether Jupyter starts in the repo folder or in notebooks/.
project_root = Path.cwd()
if not (project_root / 'src').is_dir():
    project_root = project_root.parent
os.chdir(project_root)
sys.path.insert(0, str(project_root))

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch

from src.train import MAX_EPOCHS, train
from src.utils import FIGURES_DIR, METRICS_CSV, get_device
```

```
print('Device:', get_device())
if torch.cuda.is_available():
    print('GPU:', torch.cuda.get_device_name(0))
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
2 to 14	Finds the repository folder from either the repo folder or the `notebooks/` folder, changes to it and adds it to the import path.	Working folder	Imports and relative paths then work from both places.
16 to 21	Imports plotting, numbers, tables and PyTorch.	Libraries	
23 and 24	Imports `train`, `MAX_EPOCHS`, the figure folder, the results file and `get_device`.	Reuse	The notebook calls the same function as the rest of the project.
26 and 27	Prints the device and the GPU name.	Check	Shows that the run is on the GPU (on the RTX 3050 here).

Settings cell (id 8fc2b7e8)

```
MODELS_TO_COMPARE = ['scratch_cnn', 'mobilenet_v2_full']   # our own CNN vs the best transfer model on validation
FRACTIONS = [0.05, 0.10, 0.25, 0.50, 1.00]              # share of the training images
SEEDS = [0, 1, 2]                                       # seeds 3 and 4 (the CPU laptops) can be added later
TRAIN_SIZE = 18900                                      # 70% of 27,000 images
SIZES = [round(fraction * TRAIN_SIZE) for fraction in FRACTIONS]

print('Training sizes:', SIZES)
print('Runs in total:', len(MODELS_TO_COMPARE) * len(FRACTIONS) * len(SEEDS), '| epochs per run: up to', MAX_EPOCHS)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1	Two models: our scratch CNN and MobileNetV2 fully fine-tuned.	Comparison	The best transfer model on validation (98.22% against 97.69% for ResNet-18 full).
2	Five shares of the training images.	Fraction	5%, 10%, 25%, 50% and 100%.
3	Three seeds.	Seeds	Several seeds show whether a difference is larger than the random spread.
4	Hard-coded size of the training split.	Constant	Equal to 70% of 27,000. The notebook does not recompute it.
5	Turns each share into a number of images: 945, 1890, 4725, 9450, 18900.	Derived constants	Used for the x-axis and the table.
7 and 8	Prints the sizes, and the number of runs (2 x 5 x 3 = 30) and the epoch limit.	Output	30 runs in total, including the 6 finished 100% runs from Parts B and C.

Sweep cell (id ab16d926)

```
def already_done(model, fraction, seed):
    '''True if metrics.csv already has a row for this (model, fraction, seed).'''
    if not METRICS_CSV.exists():
        return False
    table = pd.read_csv(METRICS_CSV)
    same = (table['model'] == model) & np.isclose(table['fraction'], fraction) & (table['seed'] == seed)
    return bool(same.any())


jobs = [(model, fraction, seed) for seed in SEEDS for model in MODELS_TO_COMPARE for fraction in FRACTIONS]
sweep_started = time.time()
for model, fraction, seed in jobs:
    if already_done(model, fraction, seed):
        print(f'skip  {model:18s} fraction {fraction:<5g} seed {seed}: already in metrics.csv')
        continue
    started = time.time()
    with contextlib.redirect_stdout(io.StringIO()):   # hide the per-epoch lines
        result = train(model, fraction, seed)
    print(f'done  {model:18s} fraction {fraction:<5g} seed {seed}: test accuracy {result["accuracy"]:.4f}, '
          f'macro-F1 {result["macro_f1"]:.4f}, best epoch {result["best_epoch"]:2d}, '
```

```
          f'{(time.time() - started) / 60:.1f} min')
print(f'Sweep finished in {(time.time() - sweep_started) / 60:.1f} minutes.')
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1 to 7	`already_done` says True if `metrics.csv` already has this (model, fraction, seed).	Resumable check	Makes the notebook safe to stop and restart.
4 and 5	If the file does not exist yet, nothing is done.	Edge case	
6 and 7	Reads the table, builds a True/False mask for the three conditions, and returns True if any row matches.	Boolean mask, `np.isclose`	`np.isclose` is used so 0.05 matches 0.05 even with binary rounding.
10	Builds the list of all 30 jobs. Seed is the outer loop, then model, then fraction.	Ordering	If the notebook stops early, every model has the same seeds done. Each seed is complete before the next.
11	Starts the sweep timer.	Timing	
12 to 15	For each job: if already done, print "skip" and move on.	Resumable loop	The 100% rows from Parts B and C are skipped.
17	Starts a timer for this run.	Timing	
18 and 19	Calls `train()` and hides its per-epoch printing with `redirect_stdout`.	`contextlib.redirect_stdout`	Keeps the notebook output short: one line per run.
20 to 23	Prints one summary line with accuracy, macro-F1, best epoch and minutes.	Output	The best epoch is shown so a run that stopped at epoch 20 can be spotted.
24	Prints the total time.	Output	About 20 minutes on the RTX 3050, from the log.

Results table cell (id c9cec3c1)

```
table = pd.read_csv(METRICS_CSV)
table = table[table['model'].isin(MODELS_TO_COMPARE)].copy()
table['n_images'] = (table['fraction'] * TRAIN_SIZE).round().astype(int)

summary = (table.groupby(['model', 'n_images'])
           .agg(seeds=('seed', 'count'),
                accuracy_mean=('accuracy', 'mean'), accuracy_sd=('accuracy', lambda s: s.std(ddof=1)),
                f1_mean=('macro_f1', 'mean'), f1_sd=('macro_f1', lambda s: s.std(ddof=1)))
           .reset_index())

show = summary.copy()
show['test accuracy (%)'] = [f'{100 * m:.2f} ± {100 * (0 if pd.isna(s) else s):.2f}'
                             for m, s in zip(show['accuracy_mean'], show['accuracy_sd'])]
show['test macro-F1 (%)'] = [f'{100 * m:.2f} ± {100 * (0 if pd.isna(s) else s):.2f}'
                             for m, s in zip(show['f1_mean'], show['f1_sd'])]
display(show[['model', 'n_images', 'seeds', 'test accuracy (%)', 'test macro-F1 (%)']])
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1 and 2	Reads the table and keeps only the two models.	Filtering	
3	Adds a column with the number of training images from the fraction.	Derived column	Fraction 0.05 becomes 945 images. The `round` avoids 944.99 type errors.
5 to 9	Groups by model and size, and computes per group: number of seeds, mean and sample SD (`ddof=1`) of accuracy and macro-F1.	Group-by aggregation, mean ± SD	`ddof=1` divides by the number of seeds minus 1, which is right for a small sample. With one seed the SD is NaN.

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
11 to 15	Makes display copies and text strings of the form "mean ± SD" in percent. A NaN SD (one seed) is shown as 0.	Formatting	Displays 93.69 ± 0.20.
16	Shows the table.	Output	The numbers in the README come from this table.

Plot cell (id 2ee2e351)

```
labels = {'scratch_cnn': 'Scratch CNN (our own)', 'mobilenet_v2_full': 'MobileNetV2, fully fine-tuned'}

fig, axes = plt.subplots(1, 2, figsize=(13, 4.8))
panels = [('accuracy_mean', 'accuracy_sd', 'Test accuracy (%)'), ('f1_mean', 'f1_sd', 'Test macro-F1 (%)')]
for ax, (mean_column, sd_column, title) in zip(axes, panels):
    for model in MODELS_TO_COMPARE:
        rows = summary[summary['model'] == model].sort_values('n_images')
        ax.errorbar(rows['n_images'], 100 * rows[mean_column], yerr=100 * rows[sd_column].fillna(0),
                    marker='o', capsize=4, linewidth=2, label=labels[model])
    ax.set_xscale('log')
    ax.set_xticks(SIZES)
    ax.set_xticklabels([f'{n:,}\n({f:.0%})' for n, f in zip(SIZES, FRACTIONS)])
    ax.minorticks_off()
    ax.set_xlabel('Labelled training images (log scale)')
    ax.set_ylabel(title)
    ax.grid(alpha=0.3)
    ax.legend(loc='lower right')
fig.suptitle('Data efficiency on EuroSAT: test score vs. amount of labelled training data (mean ± sample SD over seeds)')
fig.tight_layout()
fig.savefig(FIGURES_DIR / 'data_efficiency.png', dpi=160)
plt.show()
print('Saved', FIGURES_DIR / 'data_efficiency.png')
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1	Friendly names for the legend.	Labels	
3	Makes two plot panels side by side: accuracy and macro-F1.	Subplots	One figure, two measures, same x-axis.
4	The two panels are described as (mean column, SD column, title).	Data-driven plot	One loop draws both.
6 to 9	For each model draws its line with dots and error bars. Values are multiplied by 100 to show percent; NaN SD becomes 0.	Error bars, mean ± SD	The bar length is one sample SD.
10	Uses a logarithmic x-axis.	Log scale	The sizes grow by 2 to 2.5 times each step, so a log axis spaces them evenly. A linear axis would crowd the small sizes.
11	Sets the x tick positions to the five sizes.	Ticks	
12	Labels each tick with the number of images and the percent.	Tick labels	For example "945 (5%)".
13	Turns off the minor ticks.	Plot style	The log axis would otherwise add many small ticks.
14 to 18	Axis labels, a grid and the legend.	Plot style	
19 and 20	Adds a title, fits the layout and saves the figure at 160 dpi.	Figure output	The file `results/figures/data_efficiency.png` is in git.
21 and 22	Shows the figure and prints the saved path.	Output	

Gap cell (id b9596086)

```
pivot = summary.pivot(index='n_images', columns='model', values='accuracy_mean') * 100
gap = pd.DataFrame({'scratch CNN (%)': pivot['scratch_cnn'], 'MobileNetV2 full (%)': pivot['mobilenet_v2_full']})
gap['gap (points)'] = gap['MobileNetV2 full (%)'] - gap['scratch CNN (%)']
display(gap.round(2))
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1	Turns the summary into a table: one row per size, one column per model, as percent.	Pivot	
2	Puts the two models into a table with clear names.	Table	
3	The gap is MobileNetV2 minus scratch, in percentage points.	Gap	Shows how much transfer learning helps at each size.
4	Shows the rounded table.	Output	The gap goes from 16.62 at 945 images to 5.56 at 18,900.

Text cells. The written interpretation (for example "transfer learning helps most when labels are scarce") and the list of study limits are text, not code. The limits are: the same epoch budget for all sizes, MobileNetV2 chosen on the full data and not tuned per size, only 3 seeds so far, and a random split that can place neighbouring patches in both train and test.

4.5 `check_setup.py` (every function)

Why this file exists: each person runs it on their laptop before the project starts. It checks Python, packages, Git, the GPU, PyTorch, disk space and speed. It can also download the data and the pretrained weights. It writes a short report to `docs/setup/<name>.md`, and prints one line for the team chat.

Module docstring and constants (lines 1 to 39)

The docstring explains the usage and the six checks. The constants that matter are:

```
ROOT = Path(__file__).resolve().parent
RESULTS = []  # each entry: (check, status, detail, fix)
INFO = {}     # values used for the summary line

PACKAGES = [
    "torch", "torchvision", "numpy", "pandas", "matplotlib", "scikit-learn",
    "pillow", "fastapi", "uvicorn", "python-multipart", "notebook", "ipykernel",
]
TRAIN_IMAGES = 18900  # 70% of 27,000
BATCH = 64
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
30	The repository folder, found from the file's place.	Path relative to the file	Same reason as in `utils.py`.
31	List of results. Each check adds one tuple: check, status, detail, fix.	Global list	The summary and the saved file are built from this list.
32	Dictionary of values for the team-chat line (Python version, torch, GPU, verdict).	Global dictionary	
34 to 37	The packages that must be installed.	Dependency check	Torch and torchvision are listed here but installed by a separate command.
38 and 39	Training image count (70% of 27,000) and batch size 64.	Constants	Used for the speed estimate.

`record` (lines 42 to 48)

```
def record(check, status, detail, fix=""):
    """Store one result and print it."""
    RESULTS.append((check, status, detail, fix))
    line = f"  [{status}] {check}: {detail}"
    if fix and status != "OK":
        line += f"\n         fix: {fix}"
    print(line)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
43	Stores the result in the list.	Accumulator	
44	Builds the printed line, for example `[OK] Python: 3.12.x`.	String formatting	
45 and 46	If the result is not OK and there is a fix, shows the fix under it.	One-line fix	Each message tells the person what to do.
47	Prints the line.	Output	

`run_check` (lines 51 to 58)

```
def run_check(title, func):
    """Run one check. If it crashes, record FAIL and keep going."""
    print(f"\n== {title}")
    try:
        func()
    except Exception as err:  # never stop the other checks
        record(title, "FAIL", f"unexpected error: {type(err).__name__}: {err}",
               "send this line to Nitin")
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
52	Prints a heading for the check.	Output	
53 to 55	Calls the check function.	Function as argument	The checks are passed in one by one from `main`.
56 and 57	If anything crashes, records a FAIL with the error and continues.	Exception handling	One broken check must not stop the others. The message asks the person to send it to Nitin.

`run_cmd` (lines 61 to 67)

```
def run_cmd(args, timeout=10):
    """Run a command and return its text output, or '' if it fails."""
    try:
        out = subprocess.run(args, capture_output=True, text=True, timeout=timeout)
        return out.stdout.strip() if out.returncode == 0 else ""
    except (OSError, subprocess.SubprocessError):
        return ""
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
64	Runs a program and captures its text output, with a time limit of 10 seconds.	`subprocess`	A hung program cannot block the check forever.
65	Returns the output if the program succeeded, otherwise an empty string.	Return code	An empty string means "not found" for the callers.
66 and 67	If the program does not exist or times out, returns an empty string.	Error handling	`git` or `nvidia-smi` may simply be missing.

`check_python` (lines 72 to 83)

```
def check_python():
    v = sys.version_info
    ver = f"{v.major}.{v.minor}.{v.micro}"
    INFO["python"] = ver
```

```
    if (v.major, v.minor) == (3, 12):
        record("Python", "OK", ver)
    elif (v.major, v.minor) in [(3, 10), (3, 11), (3, 13)]:
        record("Python", "WARN", f"{ver} - team standard is 3.12",
               "install Python 3.12 from python.org and make a new .venv with it")
    else:
        record("Python", "FAIL", f"{ver} is not supported",
               "install Python 3.12 from python.org and make a new .venv with it")
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
73 to 75	Reads the running Python version and stores it for the summary.	`sys.version_info`	
76 and 77	Python 3.12 is OK (the team standard).	Standard	
78 and 79	3.10, 3.11 and 3.13 are WARN (may work, not the team standard).	Warning	
80 to 83	Any other version is FAIL.	Hard requirement	

`check_packages` (lines 86 to 95)

```
def check_packages():
    for name in PACKAGES:
        try:
            record(name, "OK", importlib.metadata.version(name))
        except importlib.metadata.PackageNotFoundError:
            if name in ("torch", "torchvision"):
                fix = "install PyTorch with the command in README, Setup step 4"
            else:
                fix = "pip install -r requirements.txt"
            record(name, "FAIL", "MISSING", fix)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
87	Goes through each package name.	Loop	
88 and 89	Asks the installed package list for its version and records OK.	`importlib.metadata`	Works without importing the package, so it is fast.
90 to 95	If the package is missing, FAIL with the right fix: torch needs the special PyTorch command, others need `requirements.txt`.	Exception, specific fix	Torch is installed with a CUDA-specific command, not with `pip install -r`.

`check_git` (lines 98 to 114)

```
def check_git():
    version = run_cmd(["git", "--version"])
    if not version:
        record("Git", "FAIL", "git not found", "install Git from git-scm.com, then reopen PowerShell")
        return
    record("Git", "OK", version)
    name = run_cmd(["git", "config", "user.name"])
    email = run_cmd(["git", "config", "user.email"])
    record("Git user.name", "OK" if name else "WARN", name or "not set",
           'git config --global user.name "Your Name"')
    if email:
        record("Git user.email", "OK", email)
    else:
        record("Git user.email", "WARN", "not set",
               'git config --global user.email "you@example.com"')
    print("  REMINDER: this must be the same email as your GitHub account,")
    print("            or your commits will not count as yours.")
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
99 to 103	Checks that Git runs. If not, FAIL and stop this check.	Required tool	
104 and 105	Reads the Git user name and email.	Git config	Commits are linked to a person by these values.
106 to 108	Name missing is a WARN with the command to set it.	Warning	
109 to 113	Email is OK or WARN with the command.	Warning	
114 and 115	Prints a reminder that the email must match GitHub.	Output	Otherwise commits do not count for the person in GitHub's statistics.

`check_nvidia` (lines 117 to 130)

```
def check_nvidia():
    # nvidia-smi is normally on PATH; on some laptops it is only in the NVIDIA folder
    out = ""
    for exe in ["nvidia-smi", r"C:\Program Files\NVIDIA Corporation\NVSMI\nvidia-smi.exe"]:
        out = run_cmd([exe, "--query-gpu=name,driver_version,memory.total",
                       "--format=csv,noheader"])
        if out:
            break
    if not out:
        record("NVIDIA GPU", "WARN", "no NVIDIA GPU or driver found",
               "the project still runs on CPU, but training will be slow")
        return
    INFO["gpu_line"] = out.splitlines()[0]
    record("NVIDIA GPU", "OK", INFO["gpu_line"])
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
119 to 124	Tries `nvidia-smi` on the PATH, then the default NVIDIA install folder. Stops at the first that works.	Fallback search	On some laptops the tool is not on the PATH.
125 to 128	No GPU found is a WARN, not a FAIL, because the project still runs on CPU.	Warning	
129 and 130	Stores the first line (GPU name, driver version, memory) and records OK.	Output	

`check_torch` (lines 133 to 158)

```
def check_torch():
    try:
        import torch
    except ImportError:
        record("PyTorch", "FAIL", "torch is not installed",
               "install PyTorch with the command in README, Setup step 4")
        return
    cuda_build = torch.version.cuda or "none (CPU-only build)"
    cuda_ok = torch.cuda.is_available()
    INFO["torch"] = f"{torch.__version__} + CUDA {cuda_build}"
    record("PyTorch", "OK", f"version {torch.__version__}, CUDA build {cuda_build}, "
                            f"cuda available = {cuda_ok}")
    if cuda_ok:
        name = torch.cuda.get_device_name(0)
        vram = torch.cuda.get_device_properties(0).total_memory / 1024 ** 3
        INFO["gpu"] = f"{name}, {vram:.1f} GB VRAM"
        record("PyTorch GPU", "OK", INFO["gpu"])
    elif "gpu_line" in INFO:
        INFO["gpu"] = "CPU only (CPU-only PyTorch installed)"
        record("PyTorch GPU", "FAIL", "nvidia-smi found a GPU but PyTorch cannot use it",
               "you installed the CPU-only PyTorch; reinstall using the CUDA command from "
               "https://pytorch.org/get-started/locally/ (Windows, Pip, CUDA)")
    else:
        INFO["gpu"] = "CPU only"
```

```
        record("PyTorch GPU", "WARN", "no GPU for PyTorch, will use CPU",
               "fine for this check; training needs a GPU")
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
134 to 139	Imports PyTorch. If missing: FAIL with the install hint.	Import check	
140	Reads the CUDA version PyTorch was built with, or says it is CPU-only.	`torch.version.cuda`	
141	Asks if PyTorch can use the GPU.	`torch.cuda.is_available()`	
142 to 144	Records the PyTorch version and CUDA build.	Output	
145 to 149	If the GPU works: reads its name and memory and records OK.	`get_device_properties`	VRAM is the GPU's own memory.
150 to 154	nvidia-smi sees a GPU but PyTorch does not: this is the CPU-only PyTorch problem. FAIL with the reinstall link.	Diagnosis	The most common setup problem on Windows (see the README).
155 to 158	No GPU at all: WARN, the check still passes.	Warning	

`check_disk` (lines 161 to 167)

```
def check_disk():
    free_gb = shutil.disk_usage(ROOT).free / 1024 ** 3
    if free_gb < 5:
        record("Disk space", "WARN", f"only {free_gb:.1f} GB free",
               "free some space: the dataset and PyTorch need several GB")
    else:
        record("Disk space", "OK", f"{free_gb:.1f} GB free")
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
162	Reads the free space on the drive of the repository, in GB.	`shutil.disk_usage`	
163 to 167	Under 5 GB is a WARN, otherwise OK.	Threshold	The dataset and PyTorch need several GB.

`make_small_cnn` (lines 170 to 179)

```
def make_small_cnn(torch):
    """Small scratch CNN: 3 blocks of Conv-BatchNorm-ReLU-MaxPool, then a Linear to 10 classes."""
    nn = torch.nn

    def block(cin, cout):
        return nn.Sequential(nn.Conv2d(cin, cout, 3, padding=1), nn.BatchNorm2d(cout),
                             nn.ReLU(), nn.MaxPool2d(2))

    return nn.Sequential(block(3, 32), block(32, 64), block(64, 128),
                         nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Linear(128, 10))
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
171 to 172	Gets the `nn` module from the torch object passed in.	Dependency passed in	The function does not import torch at the top, so the check can fail softly.
174 to 176	A block is convolution, BatchNorm, ReLU and MaxPool.	Convolution block	The same pattern as Tejash's baseline.
178 and 179	Three blocks (32, 64, 128 filters), then pooling, flatten and a linear layer to 10 classes.	Network	It is a benchmark model, not the real project CNN. It is built here to measure speed.

`sync` (lines 182 to 185)

```
def sync(torch, device):
    """GPU runs are asynchronous: wait for them to finish before reading the clock."""
    if device.type == "cuda":
        torch.cuda.synchronize()
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
183 to 185	On a GPU, waits until all queued work has finished.	`torch.cuda.synchronize`	The GPU runs in the background. Without waiting, the timer would measure too little time.

`benchmark` (lines 188 to 209)

```
def benchmark(torch, model, device, steps, warmup=5):
    """Train on random 64x64 data and return images per second."""
    model = model.to(device)
    model.train()
    opt = torch.optim.Adam(model.parameters())
    loss_fn = torch.nn.CrossEntropyLoss()
    x = torch.rand(BATCH, 3, 64, 64, device=device)
    y = torch.randint(0, 10, (BATCH,), device=device)

    def one_step():
        opt.zero_grad()
        loss_fn(model(x), y).backward()
        opt.step()

    for _ in range(warmup):
        one_step()
    sync(torch, device)
    start = time.perf_counter()
    for _ in range(steps):
        one_step()
    sync(torch, device)
    return steps * BATCH / (time.perf_counter() - start)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
190 to 192	Moves the model to the device and puts it in training mode.	Training step	A speed test must train, not only score.
193 and 194	Makes Adam and cross-entropy, as in real training.	Optimiser, loss	
195 and 196	Makes random images and random labels with batch size 64.	Synthetic data	Speed does not depend on the real pictures.
198 to 201	One training step: clear gradients, forward, loss, backward, update.	Training step	The same five steps as `train()`.
203 and 204	Runs 5 warm-up steps and waits.	Warm-up	The first steps are slower (memory setup, GPU start-up). They must not be timed.
205 to 209	Times the steps and returns images per second.	Throughput	Images per second = steps times batch size divided by seconds.

`check_speed` (lines 212 to 250)

```
def check_speed():
    try:
        import torch
        import torchvision
    except ImportError:
        record("Speed test", "FAIL", "torch or torchvision missing, cannot test",
               "install PyTorch first (README, Setup step 4)")
        return
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    steps = 30 if device.type == "cuda" else 5
```

```
    print(f"  running on {device}: {steps} timed steps per model, batch {BATCH}, random data")
    models = {
        "Small CNN": lambda: make_small_cnn(torch),
        "ResNet-18": lambda: torchvision.models.resnet18(weights=None, num_classes=10),
    }
    for name, build in models.items():
        if device.type == "cuda":
            torch.cuda.reset_peak_memory_stats()
        try:
            ips = benchmark(torch, build(), device, steps)
        except torch.cuda.OutOfMemoryError:
            torch.cuda.empty_cache()
            record(f"Speed {name}", "WARN", f"batch {BATCH} does not fit, use 32",
                   "set the batch size to 32 in train.py")
            if name == "ResNet-18":  # no timing possible, so give the cautious verdict
                INFO["resnet_sec"] = "out of memory"
                verdict(float("inf"))
            continue
        sec_epoch = TRAIN_IMAGES / ips
        detail = (f"{ips:.0f} images/s, {sec_epoch:.0f} s per epoch, "
                  f"{sec_epoch * 20 / 60:.1f} min for 20 epochs")
        if device.type == "cuda":
            peak = torch.cuda.max_memory_allocated() / 1024 ** 3
            detail += f", peak VRAM {peak:.2f} GB"
        record(f"Speed {name}", "OK", detail)
        if name == "ResNet-18":
            INFO["resnet_sec"] = f"{sec_epoch:.0f}"
            verdict(sec_epoch)
    print("  NOTE: these are lower bounds, because real runs also load and augment images.")
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
213 to 219	Imports torch and torchvision. If missing: FAIL and stop.	Import check	
220 and 221	Picks the device. Uses 30 timed steps on a GPU and only 5 on a CPU.	Time budget	A CPU is much slower, so fewer steps keep the check short.
222	Prints what is being tested.	Output	
223 to 226	Two models: a small CNN and ResNet-18 (random weights, 10 classes).	Benchmark models	ResNet-18 is the verdict model. Random weights are fine, because speed does not depend on the weights.
227	Goes through each model.	Loop	
228 and 229	On a GPU, resets the peak memory counter.	Memory counter	Needed to report peak VRAM later.
230 and 231	Runs the benchmark for this model and gets images per second.	`benchmark`	
232 to 239	If the GPU runs out of memory: WARN, free the cache, and for ResNet-18 give the cautious verdict.	Out-of-memory handling	The advice is "use batch 32", which is a manual change. The check does not change `train.py` itself, so this advice is for the person to act on.
240 and 241	Converts images per second into seconds per epoch on the 18,900 training images, and the 20-epoch time.	Projection	Images per second is a lower bound: real runs also load and augment images.
243 to 245	On a GPU adds the peak VRAM used.	Memory	
246 to 249	Records OK. For ResNet-18 stores the seconds and gives the verdict.	Verdict input	
250	Prints the note that these numbers are lower bounds.	Honesty note	

`verdict` (lines 253 to 262)

```
def verdict(sec_epoch):
    """Turn ResNet-18 seconds per epoch into a plain advice line."""
    if sec_epoch <= 60:
        text = "GPU-ready: can own a training part (B, C or D)"
    elif sec_epoch <= 300:
        text = "OK for small training runs (B)"
    else:
        text = "Take a non-training part (A or E), or train on a teammate's GPU"
    INFO["verdict"] = text
    record("Verdict", "OK", text)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
255 and 256	Up to 60 seconds per epoch: GPU-ready, can own a training part.	Threshold	
257 and 258	60 to 300 seconds: OK for small runs.	Threshold	
259 and 260	Above 300 seconds: take a non-training part, or use a teammate's GPU.	Threshold	The thresholds are a team judgement, not a law.
261 and 262	Stores the text for the team line and records it.	Output	

`check_download` (lines 265 to 289)

```
def check_download():
    from torchvision import datasets, models
    from torchvision.models import MobileNet_V2_Weights, ResNet18_Weights

    data_dir = ROOT / "data"
    data_dir.mkdir(exist_ok=True)
    try:
        ds = datasets.EuroSAT(root=str(data_dir), download=True)
        n, k = len(ds), len(ds.classes)
        ok = n == 27000 and k == 10
        record("EuroSAT dataset", "OK" if ok else "FAIL", f"{n} images, {k} classes",
               "" if ok else "delete the data/eurosat folder and run --download again")
    except Exception as err:
        record("EuroSAT dataset", "FAIL", f"download failed: {err}",
               "check your internet connection and run --download again")
    for label, build in [
        ("ResNet-18 weights", lambda: models.resnet18(weights=ResNet18_Weights.DEFAULT)),
        ("MobileNetV2 weights", lambda: models.mobilenet_v2(weights=MobileNet_V2_Weights.DEFAULT)),
    ]:
        try:
            build()
            record(label, "OK", "downloaded to the torch cache")
        except Exception as err:
            record(label, "FAIL", f"download failed: {err}",
                   "check your internet connection and run --download again")
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
266 and 267	Imports the dataset and model tools (only when `--download` is used).	Lazy import	Keeps the normal check light.
269 and 270	Makes the `data/` folder if needed.	Folder	
271 and 272	Downloads EuroSAT into `data/` (only if it is missing), then reads the count and the classes.	Dataset download	torchvision stores it in `data/eurosat/2750`.
273 to 277	Checks that there are 27,000 images and 10 classes. OK or FAIL with a fix.	Sanity check	Catches a partial download.
278 to 280	A download error becomes FAIL with a fix.	Error handling	

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
281 to 290	For each pretrained model, builds it with its default weights. This downloads the weights into the torch cache.	Pretrained weights	`build()` is a lambda, so the download happens only inside the `try`.

`save_summary` (lines 294 to 304)

```
def save_summary(name, line):
    """Write docs/setup/<name>.md, overwriting it if it exists."""
    folder = ROOT / "docs" / "setup"
    folder.mkdir(parents=True, exist_ok=True)
    safe = "".join(c for c in name.lower() if c.isalnum())
    rows = ["| Check | Status | Detail |", "|---|---|---|"]
    rows += [f"| {c} | {s} | {d} |" for c, s, d, _ in RESULTS]
    text = (f"# Setup report: {name}\n\n" + "\n".join(rows) +
            f"\n\nTeam line:\n\n    {line}\n")
    (folder / f"{safe}.md").write_text(text, encoding="utf-8")
    return folder / f"{safe}.md"
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
296 and 297	Makes the `docs/setup` folder if needed.	Folder	
298	Makes a safe file name: lower case, letters and digits only.	String cleaning	"Srivalli" becomes "srivalli".
299 to 301	Builds a Markdown table of all results.	Markdown	
302 to 304	Adds a title, the table and the team line, then writes the file in UTF-8.	File output	The file goes into git through a pull request.

`main` (lines 307 to 338)

```
def main():
    parser = argparse.ArgumentParser(description="Check this laptop for the EuroSAT project.")
    parser.add_argument("--name", required=True, help="your first name, e.g. Tejash")
    parser.add_argument("--download", action="store_true",
                        help="also download EuroSAT and the pretrained weights")
    args = parser.parse_args()
    sys.stdout.reconfigure(errors="replace")  # an odd character in an error must not crash the printing

    run_check("1. Python", check_python)
    run_check("2. Packages", check_packages)
    run_check("3. Git", check_git)
    run_check("4. NVIDIA GPU", check_nvidia)
    run_check("5. PyTorch and GPU", check_torch)
    run_check("6. Disk space", check_disk)
    run_check("7. Speed test (project score)", check_speed)
    if args.download:
        run_check("8. Download data and weights", check_download)

    print("\n========== SUMMARY ==========")
    for check, status, detail, _ in RESULTS:
        print(f"{status:5} {check:24} {detail}")

    line = (f"{args.name} | Python {INFO.get('python', '?')} | torch {INFO.get('torch', '?')} | "
            f"{INFO.get('gpu', '?')} | ResNet-18 {INFO.get('resnet_sec', '?')} s/epoch | "
            f"{INFO.get('verdict', '?')}")
    print("\nPaste this line in the team chat:")
    print(line)
    path = save_summary(args.name, line)
    print(f"\nSaved to {path}")

    failed = any(status == "FAIL" for _, status, _, _ in RESULTS)
    sys.exit(1 if failed else 0)
```

```
if __name__ == "__main__":
    main()
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
308 to 312	Reads the command line: `--name` (required) and `--download` (optional yes/no).	`argparse`	Example: `python check_setup.py --name Tejash --download`.
313	Makes printing safe for odd characters.	`reconfigure`	A strange character in an error message must not crash the report.
315 to 321	Runs the seven standard checks in order.	Ordered checks	Each check is independent, because `run_check` catches crashes.
322 and 323	Runs the download check only with `--download`.	Optional step	Downloads are large (about 150 MB), so they are not automatic.
325 to 327	Prints the summary table, one line per result.	Output	
329 to 331	Builds the team-chat line from the stored values.	Team line	Shows Python, torch, GPU, ResNet-18 seconds per epoch and the verdict.
332 to 335	Prints the line and saves the report file.	Output	
337 and 338	Exits with code 1 if any FAIL, otherwise 0.	Exit code	A script or teacher can see whether the laptop is ready. WARN does not fail.
341 and 342	Runs `main()` only when started directly.	`__main__` guard	

4.6 `app/server.py` (the website server, all of it)

Why this file exists: the team built a website (pages in `frontend/`, by Tejash). One program serves the pages and three API routes. The file header says the file has no named owner. Nitin ran it and checked it with real images (log, 2026-11-05). It runs offline and listens on this computer only.

Imports, folders and constants (lines 16 to 41)

```
FRONTEND = ROOT.parent / "frontend"                # Tejash's pages
CLASSES = ["AnnualCrop", "Forest", "HerbaceousVegetation", "Highway", "Industrial",
           "Pasture", "PermanentCrop", "Residential", "River", "SeaLake"]    # EuroSAT folder order
MAX_UPLOAD = 15_000_000                             # bytes
HEADLINE = "mobilenet_v2_full_frac1_seed0"          # the model behind the metrics page (Part E)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
16 to 35	Imports: base64 and io for the picture, threading for the lock, numpy and pandas, torch, FastAPI, CORS, file responses, static files, colour maps, Pillow and torchvision transforms. Also `build_model` from `train.py` and the paths from `utils.py`.	Libraries and reuse	`build_model` is reused, so the website uses the same model code as the training.
37	`FRONTEND` is the `frontend/` folder next to the Backend folder.	Path	`ROOT.parent` goes from `Backend/` to the repository root.
38 and 39	The ten class names in the same order as EuroSAT's folders.	Label order	The index of the output score maps to the name at the same position.
40	Upload limit 15 MB.	Limit	Stops very large uploads from filling memory.

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
41	The model used for the metrics page.	Constant	The metrics page uses the seed-0 model's confusion matrix.

`last_relu` (lines 44 to 46)

```
def last_relu(net):
    """Last ReLU of the scratch CNN = end of its last convolution block."""
    return [m for m in net if isinstance(m, nn.ReLU)][-1]
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
45	Finds the last ReLU layer in the scratch CNN.	Layer search	Grad-CAM needs a convolution layer. The last ReLU is the end of the last convolution block.

`CHOICES` (lines 49 to 54)

```
CHOICES = {
    "mobilenet_v2_full": ("MobileNetV2 (fully fine-tuned)", lambda net: net.features[-1]),
    "resnet18_full": ("ResNet-18 (fully fine-tuned)", lambda net: net.layer4[-1]),
    "scratch_cnn": ("Scratch CNN (our own, 3 blocks)", last_relu),
}
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
50 to 54	Three models the website offers. Each has a label and a function that finds the layer for Grad-CAM.	Dictionary of functions	The "last block" differs per architecture. The lambda finds it only when needed.

`TRANSFORM` (lines 56 to 60)

```
TRANSFORM = transforms.Compose([
    transforms.Resize((64, 64)),                    # the models were trained on 64 x 64 tiles
    transforms.ToTensor(),
    transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
])
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
57	Resizes any uploaded picture to 64 x 64.	Resize	The models were trained on 64 x 64 tiles, so every upload is shrunk or stretched to that size.
58	Turns the picture into a tensor with values 0 to 1.	`ToTensor`	Same as in training.
59	Normalises with the ImageNet numbers.	Normalisation	Must match training, or predictions would be wrong.

Cache and lock (lines 62 to 63)

```
_loaded = {}                                        # model id -> (network, Grad-CAM layer)
_lock = threading.Lock()                            # Grad-CAM puts hooks on the shared network: one request at a time
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
62	Keeps each loaded model, so the weights are read from disk only once.	Cache	
63	One lock for the Grad-CAM step.	Thread lock	Grad-CAM adds hooks to the shared network. Two requests at once would mix up their hooks. The price is that uploads wait in turn.

`available` (lines 66 to 67)

```
def available():
    return [i for i in CHOICES if (MODELS_DIR / f"{i}_frac1_seed0.pth").exists()]
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
67	Returns the model ids whose full-data weight file is in `models/`.	File check	The name is the same one `train()` writes (`frac1_seed0`). A missing file means the model is not offered.

`get_model` (lines 70 to 76)

```
def get_model(model_id):
    if model_id not in _loaded:
        net, _ = build_model(model_id)
        net.load_state_dict(torch.load(MODELS_DIR / f"{model_id}_frac1_seed0.pth", map_location="cpu", weights_only=True))
        net.eval()
        _loaded[model_id] = (net, CHOICES[model_id][1](net))
    return _loaded[model_id]
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
71	If the model is not in memory yet...	Lazy loading	Models load on first use.
72	Builds the empty model with `build_model` (from `train.py`).	Reuse	The architecture is the same code as in training.
73	Loads the saved weights onto the CPU, with `weights_only=True`.	`torch.load`, `load_state_dict`	`weights_only` loads only numbers and no code, which is safer for a file that might be replaced.
74	Switches to evaluation mode.	`model.eval()`	BatchNorm and dropout behave as in scoring.
75	Stores the model and its Grad-CAM layer.	Cache	
76	Returns both.	Return value	

`overlay_png` (lines 79 to 89)

```
def overlay_png(heat, size):
    """Grad-CAM map (64 x 64, values 0 to 1) -> transparent PNG with the size of the uploaded image, as a data URL."""
    width, height = size
    scale = min(1.0, 512 / max(width, height))        # keep the answer small
    size = (max(1, round(width * scale)), max(1, round(height * scale)))
    heat = np.asarray(Image.fromarray((heat * 255).astype(np.uint8)).resize(size, Image.BILINEAR)) / 255.0
    rgba = colormaps["jet"](heat)                      # (h, w, 4) floats
    rgba[..., 3] = 0.15 + 0.55 * heat                  # hot areas more opaque
    buffer = io.BytesIO()
    Image.fromarray((rgba * 255).astype(np.uint8), "RGBA").save(buffer, format="PNG")
    return "" + base64.b64encode(buffer.getvalue()).decode()
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
81 and 82	Gets the uploaded width and height, and a scale that makes the longest side at most 512 pixels.	Scaling	Keeps the answer small. A huge upload does not make a huge reply.
83	Computes the new size, at least 1 pixel.	Rounding	
84	Turns the 64 x 64 heat map into an 8-bit image, resizes it smoothly to the new size, and back to 0 to 1 decimals.	Bilinear resize	Smooth edges, so the overlay looks natural.
85	Maps each value to a colour: blue for low, red for high.	Colour map `jet`	
86	Sets the transparency: hot areas are more opaque, from 15% to 70%.	Alpha	Cold areas stay almost invisible, so the picture shows through.

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
87 and 88	Writes the coloured image as PNG into memory.	Pillow, RGBA	
89	Returns the picture as text: `...`.	Data URL	The web page can show the overlay without a second request.

`app` and CORS (lines 92 to 93)

```
app = FastAPI(title="Eyes in the Sky")
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"], allow_headers=["*"])   # API.md asks for CORS
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
92	Creates the web application.	FastAPI	
93	Allows any web page to call the API.	CORS	The API document asks for it. For a local demo it is simple. A stricter list of allowed addresses would be safer on a public server.

`api_models` (lines 96 to 98)

```
@app.get("/api/models")
def api_models():
    return {"models": [{"id": i, "label": CHOICES[i][0]} for i in available()]}
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
96	Route `GET /api/models`.	Route	The web page calls it to fill the model list.
98	Returns the available models with their labels, as JSON.	JSON response	FastAPI turns the dictionary into JSON.

`api_predict` (lines 101 to 129)

```
@app.post("/api/predict")
def api_predict(image: UploadFile = File(...), model: str = Form(None)):
    ids = available()
    if not ids:
        raise HTTPException(503, "no model weights found in Backend/models/ (see models/README.md)")
    model_id = model or ids[0]
    if model_id not in ids:
        raise HTTPException(400, f"unknown model {model_id!r}; available: {ids}")
    data = image.file.read(MAX_UPLOAD + 1)
    if len(data) > MAX_UPLOAD:
        raise HTTPException(413, "image is larger than 15 MB")
    try:
        picture = Image.open(io.BytesIO(data)).convert("RGB")
    except (UnidentifiedImageError, OSError, Image.DecompressionBombError):
        raise HTTPException(400, "the uploaded file is not an image")

    batch = TRANSFORM(picture).unsqueeze(0)
    with _lock:
        net, layer = get_model(model_id)
        heat, _ = grad_cam(net, batch, layer)
        with torch.no_grad():
            probabilities = torch.softmax(net(batch), dim=1)[0]
    top = probabilities.topk(3)
    return {
        "label": CLASSES[int(top.indices[0])],
        "confidence": float(top.values[0]),
        "top3": [{"label": CLASSES[int(i)], "prob": float(p)} for p, i in zip(top.values, top.indices)],
        "gradcam": overlay_png(heat.numpy(), picture.size),
    }
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
100	Route `POST /api/predict`.	Route	Receives an uploaded picture.
101	Arguments: the file (required) and an optional model name.	`UploadFile`, `File(...)`, `Form(None)`	Multipart form upload, the normal way for a browser to send a file.
102 to 104	If no weights exist: HTTP 503 (service not ready) with a message.	HTTP status	
105 and 106	Uses the requested model, or the first available one.	Default	
107 and 108	Unknown model name: HTTP 400 (bad request).	Input check	
109 and 110	Reads at most 15 MB + 1 byte. If there is more, HTTP 413 (too large).	Upload limit	Reading one byte extra is how the server knows the file is too large, without reading all of it.
112 to 115	Opens the picture and converts it to RGB. Any error means HTTP 400.	Pillow, error handling	`DecompressionBombError` is for files designed to exhaust memory.
117	Resizes to 64 x 64, normalises and adds a batch dimension of 1.	Preprocessing	Same steps as in training, so predictions are fair.
118	Takes the lock, so only one Grad-CAM runs at a time.	Thread lock	See the lock note above.
119	Gets the model and its Grad-CAM layer.	Cache	
120	Runs Grad-CAM for the predicted class.	Grad-CAM	Needs gradients, so it is not inside `no_grad`.
121 and 122	Scores the image with no gradients, then softmax gives probabilities.	`no_grad`, softmax	Probabilities add to 1.
123	Takes the three highest probabilities.	Top-k	
125	The top label.	Output	
126	Its confidence (probability).	Output	
127	The top three labels and probabilities.	Output	
128	The Grad-CAM picture as a data URL.	Overlay	

`api_metrics` (lines 132 to 149)

```
@app.get("/api/metrics")
def api_metrics():
    matrix = pd.read_csv(RESULTS_DIR / f"{HEADLINE}_confusion_matrix.csv", index_col=0)
    report = pd.read_csv(RESULTS_DIR / f"{HEADLINE}_classification_report.csv", index_col=0)
    runs = pd.read_csv(METRICS_CSV)
    runs = runs[runs["seed"].isin([0, 1, 2])]          # the 3 seeds every setting has (same as the README table)
    runs["fraction"] = runs["fraction"].round(2)

    def curve(model_name):
        mean = runs[runs["model"] == model_name].groupby("fraction")["accuracy"].mean() * 100
        return [round(float(mean.loc[f]), 2) for f in (0.05, 0.1, 0.25, 0.5, 1.0)]

    return {
        "accuracy": round(100 * float(np.trace(matrix.values)) / float(matrix.values.sum()), 2),
        "macro_f1": round(100 * float(report.loc["macro avg", "f1-score"]), 2),
```

```
        "confusion": matrix.values.astype(int).tolist(),
        "efficiency": {"fractions": [5, 10, 25, 50, 100], "scratch": curve("scratch_cnn"), "transfer": curve("mobilenet_v2_full")},
    }
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
133 and 134	Reads the confusion matrix and the classification report of the headline model (Rohan's files).	CSV, index column	Files from `evaluate.py`.
135 and 136	Reads the results table.	CSV	
137	Keeps seeds 0, 1 and 2 only.	Filter	Seeds 3 and 4 are not complete for every setting.
138	Rounds the fraction, so 0.1 and 0.10 match.	Rounding	Float keys can differ slightly.
140 to 142	For one model: mean accuracy per fraction, times 100, for the five fractions.	Group-by mean	Gives the three seeds' average.
145	Accuracy = sum of the diagonal divided by the sum of all cells, times 100.	Trace	That is the definition of accuracy from a confusion matrix.
146	Macro-F1 from the "macro avg" row of the report, times 100.	Macro-F1	Read from the saved report, not recomputed.
147	The full matrix as integers, for the Results tab.	Confusion matrix	
148	The efficiency curves for the scratch CNN and the MobileNetV2 transfer model.	Data efficiency	Same numbers as notebook 05.

Note: the metrics page's headline accuracy (98.27) comes from the seed-0 confusion matrix. It happens to equal the three-seed mean of 98.27 in the notebook.

Frontend routes (lines 152 to 164)

```
if FRONTEND.is_dir():
    @app.get("/")
    def home():
        return RedirectResponse("/Eyes_in_the_Sky_landing.html")

    @app.get("/app.html")
    def classifier_page(request: Request):
        # The page runs in demo mode (fake results) unless it is opened with ?api=<this server>. Add that for the user.
        if "api" not in request.query_params:
            return RedirectResponse(f"/app.html?api={str(request.base_url).rstrip('/')}")
        return FileResponse(FRONTEND / "app.html")

    app.mount("/", StaticFiles(directory=FRONTEND), name="frontend")        # last: catches every other file
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
152	Only adds these routes if the `frontend/` folder exists.	Conditional routes	The API still works without the pages.
153 and 154	The home address redirects to the landing page.	Redirect	
157 to 162	The classifier page redirects to itself with `?api=<this server's address>` if that is missing.	Redirect, query string	Without it the page runs in demo mode with fake results. Only the address is added, no personal data.
164	Serves all other files of the frontend, from the folder root.	Static files	Mounted last, so the API routes and the redirects are checked first.

Main block (lines 167 to 169)

```
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="127.0.0.1", port=8000)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
168 and 169	Starts the web server on this computer only, port 8000.	Uvicorn, `127.0.0.1`	Listening on 127.0.0.1 means other computers on the network cannot connect. A demo with real weights stays local.

Notes on the server (honest): `api_predict` loads a model the first time it is used, so the first upload is slower. The Grad-CAM overlay is coarse because the last block is only 2 x 2 cells on 64 x 64 tiles (see Part E).

4.7 `src/gradcam_sanity_check.py` (Nitin's control experiment)

Why this file exists: Rohan's Grad-CAM experiment had no control, so nobody could tell whether the heat map was meaningful. This script checks it. It takes 200 test images (the first 20 of each class in the order of `splits.csv`), hides 25% of the pixels in three ways, and measures how much the model's confidence falls.

```
IMAGES_PER_CLASS = 20
HIDE = 1024                                  # 25% of the 64 x 64 = 4096 pixels
QUADRANTS = {(0, 0): "top-left", (0, 1): "top-right", (1, 0): "bottom-left", (1, 1): "bottom-right"}


def painted(batch, flat_ids):
    out = batch.clone().reshape(1, 3, -1)
    out[:, :, flat_ids] = 0.0                # 0 after normalisation = the average colour of the training data
    return out.reshape(1, 3, 64, 64)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
24	20 images per class, so 200 in total.	Fixed sample	The rule is fixed, so the study cannot be tuned by choosing images.
25	1024 pixels = 25% of 4096.	Constant	
26	Names the four corners of the 2 x 2 grid.	Dictionary	Used in Check 1.
29	`painted` takes one image and the flat pixel indices to hide.	Function	
30	Copies the image and flattens height and width into one list of 4096 positions per colour.	`reshape`, `clone`	The copy keeps the original image untouched.
31	Sets the chosen positions to 0 in all three colours.	Masking	The comment says 0 is the average colour of the training data. Strictly, it is the ImageNet mean used for normalisation, which is close to but not exactly the EuroSAT average.
32	Reshapes back to a 64 x 64 image.	Reshape	

Note on the comment in line 31: the zero value is the ImageNet average colour after normalisation. That is a close approximation of the EuroSAT average, not the same thing. This is a small inaccuracy in the comment, not in the result.

```
def main():
    model, _ = build_model("mobilenet_v2_full")
    state = torch.load(MODELS_DIR / "mobilenet_v2_full_frac1_seed0.pth", map_location="cpu", weights_only=True)
    model.load_state_dict(state)
    model.eval()

    data = EuroSATDataset("test")
    generator = torch.Generator().manual_seed(0)
    rows = []
    for label in range(10):
        for index in data.df.index[data.df["label"] == label][:IMAGES_PER_CLASS]:
            batch = data[int(index)][0].unsqueeze(0)
```

```
            heat, predicted = grad_cam(model, batch, model.features[-1])
            h = heat.numpy()
            quarters = np.array([[h[:32, :32].mean(), h[:32, 32:].mean()], [h[32:, :32].mean(), h[32:, 32:].mean()]])
            flat = heat.flatten()
            top, left = torch.randint(0, 33, (2,), generator=generator).tolist()
            square = ((torch.arange(32)[:, None] + top) * 64 + (torch.arange(32)[None, :] + left)).flatten()
            patches = {"hottest": flat.topk(HIDE).indices, "coldest": (-flat).topk(HIDE).indices, "random_square": square}
            with torch.no_grad():
                before = torch.softmax(model(batch), dim=1)[0, predicted].item()
                row = {"path": data.df.loc[index, "path"], "class_name": data.df.loc[index, "class_name"],
                       "hottest_quadrant": QUADRANTS[np.unravel_index(quarters.argmax(), (2, 2))]}
                for name, ids in patches.items():
                    probs = torch.softmax(model(painted(batch, ids)), dim=1)[0]
                    row[f"drop_{name}"] = before - probs[predicted].item()
                    row[f"changed_{name}"] = int(probs.argmax().item() != predicted)
            rows.append(row)

    table = pd.DataFrame(rows)
    table.to_csv(RESULTS_DIR / "gradcam_sanity_check.csv", index=False)

    n = len(table)
    print(f"Images tested: {n}")
    print("Check 1 - hottest quadrant of the Grad-CAM map:")
    for name in QUADRANTS.values():
        count = int((table["hottest_quadrant"] == name).sum())
        print(f"  {name:13s} {count:3d}  ({count / n:.0%})")
    print("Check 2 - hide 25% of the pixels:")
    for name in ("hottest", "coldest", "random_square"):
        print(f"  {name:14s} mean confidence drop {table[f'drop_{name}'].mean():.3f} | prediction changed for
{int(table[f'changed_{name}'].sum())} of {n}")
    hot, cold = table["drop_hottest"], table["drop_coldest"]
    print(f"  hottest hurt MORE than coldest for {int((hot > cold).sum())} images, LESS for {int((hot < cold).sum())}")


if __name__ == "__main__":
    main()
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
36 to 39	Builds the MobileNetV2 model, loads its saved full-data weights on the CPU, and switches to evaluation mode.	Checkpoint loading	The check uses the saved model, not a retrained one.
41	Loads the test images with the normal test transform.	Test set	Used for analysis only. Nothing is trained.
42	A seeded generator, so the random square positions repeat.	Seed	
44 and 45	Goes through the classes 0 to 9 and takes the first 20 test images of each.	Fixed sample	Same rule each run.
46	Gets one image as a batch of 1.	Batch of one	Grad-CAM and the model expect a batch dimension.
47	Runs Grad-CAM on the last block of MobileNetV2 and gets the heat map and the predicted class.	Grad-CAM	`model.features[-1]` is the last block.
49	Average heat in each of the four quarters of the 64 x 64 map.	Quadrants	Check 1.
50	Flattens the heat map to 4096 values.	Flatten	
51 and 52	Picks a random top-left corner from 0 to 32 and builds the 32 x 32 square of pixel positions.	Random square	The square is the same size as the solid hide, so only the shape is random.
53	Three patches: the 1024 hottest pixels, the 1024 coldest pixels, and the random square (1024 pixels).	Top-k	Same size in all three, which makes the comparison fair.
54 and 55	Without gradients, reads the original probability of the predicted class.	`no_grad`, softmax	

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
56 and 57	Records the image path, the class and the hottest quadrant.	Record	
58 to 61	For each patch: hides it, reads the new probability, records the fall in confidence and whether the predicted class changed.	Deletion test	A bigger fall means the hidden area mattered more.
62	Adds the row.	Accumulate	
65 and 66	Saves one row per image to `results/gradcam_sanity_check.csv`.	Output file	
68 to 72	Prints the count of quadrants, with percentages.	Output	Gives 160 bottom-right, 36 top-right, 4 bottom-left and 0 top-left.
73 to 75	Prints the mean fall in confidence for each patch, and how many predictions changed.	Output	Gives 0.064 (hottest), 0.061 (coldest), 0.089 (random square).
76 and 77	Counts how often the hottest hide hurt more than the coldest hide.	Paired comparison	
80 and 81	Runs `main` only when started directly.	`__main__` guard	Run it with `python -m src.gradcam_sanity_check`.

The numbers in this section were checked against the saved `results/gradcam_sanity_check.csv` (200 rows): the quadrant counts and the three mean drops match the log.

4.8 `.gitignore` and `requirements.txt` (briefly)

`.gitignore` tells git which files never to commit. Important lines: `.venv/` (the virtual environment), `__pycache__/`, `data/*` (the dataset, except the README and `splits.csv`), `models/*` (weights, except the README), `*.pth` and `*.pt` (any weight file), `checkpoints/`, and `.october/` and `.claude/` (local tool settings). Keeping weights and data out of git keeps the repository small. The price is that every person must download the weights separately (see `models/README.md`).

`requirements.txt` lists the pinned versions of the other packages (numpy, pandas, matplotlib, scikit-learn, pillow, fastapi, uvicorn, python-multipart, notebook, ipykernel). PyTorch and torchvision are not in this file, because the README installs them with a special command for CPU or CUDA. Pinning means that everybody gets the same version, so results do not change because a package was updated.

---

5. How it works end to end: `train('mobilenet_v2_full', 1.0, 0)`

Assume the person has run `check_setup.py`, the data is in `data/eurosat`, and the GPU is available.

1. **The call.** `train` receives `model_name="mobilenet_v2_full"`, `fraction=1.0`, `seed=0`. It uses the defaults: `epochs=20`, `metrics_csv=results/metrics.csv`, `save_weights=None`.
2. **Check the fraction.** `0 < 1.0 <= 1` is True, so no error.
3. **Pick the device.** `get_device()` returns `"cuda"`.
4. **Set the seed.** `set_seed(0)` seeds Python, numpy and PyTorch, and sets cuDNN to deterministic. From here on, every random step repeats.
5. **Load the images (first call in this session only).** `_load_all_images("cuda")` reads `splits.csv`, opens all 27,000 pictures, stacks them into one `uint8` block of shape (27000, 3, 64, 64), and puts it on the GPU with

the labels and the path-to-row lookup. It prints the "about 20 seconds" message. Later calls in the same session return the cached block immediately.

6. **Choose the rows.** `_rows` asks `get_loader` for the train split with `fraction=1.0` (so all 18,900 rows; no subset), and for validation and test (4,050 each). It turns the paths into row numbers on the GPU.
7. **Build the model.** `build_model("mobilenet_v2_full")` finds `("mobilenet_v2", False)` in `TRANSFER`. Because `freeze` is False, it calls `models.build_transfer_model("mobilenet_v2", False)`. That loads ImageNet weights (from the torch cache, or downloaded the first time), leaves every layer trainable, and replaces the 1000-class output with a new 10-class linear layer. The learning rate is `1e-4`.
8. **Prepare the training.** The model moves to the GPU. Loss is `CrossEntropyLoss`. Adam gets every parameter (all have `requires_grad` True) with `lr=1e-4` and `foreach=False`. A seeded generator is made for the shuffle.
9. **Epoch 1.** `model.train()`. The training rows are shuffled with the seeded generator. The rows are cut into 296 batches of 64 (the last one has 20 images). For each batch: `_prepare` (bytes to 0 to 1, random 90-degree turns and flips per image, normalisation), `zero_grad`, forward pass, loss, `backward`, `step`, and the running totals are updated on the GPU.
10. **Validate.** `_score` puts the model in `eval()`, scores the 4,050 validation images in groups of 256 without gradients, and returns loss and accuracy. The four history numbers are stored, and one line is printed with the epoch number, losses, accuracies and seconds.
11. **Best so far?** If the validation accuracy is above the best so far, the epoch number and a CPU copy of all weights are kept.
12. **Repeat steps 9 to 11** for epochs 2 to 20. Each epoch reshuffles the training rows, and the seeded generator makes the shuffles repeatable.
13. **Restore the best.** After epoch 20, `load_state_dict(best_weights)` puts back the weights of the best validation epoch (not necessarily epoch 20).
14. **Test once.** `_score` scores the 4,050 test images with those weights. Accuracy comes from the scores, and macro-F1 comes from `sklearn.metrics.f1_score(..., average="macro")` on the predicted and true labels.
15. **Write the row.** A dictionary `{model, fraction, seed, accuracy, macro_f1}` is made. `_save_row` reads `results/metrics.csv`, removes any old row with the same model, fraction and seed, appends the new row, and writes the file with `\n` line endings.
16. **Save the weights.** `save_weights` is None, and `fraction == 1.0`, so it is True. `models/` is created if needed, and the best state dictionary is saved as `models/mobilenet_v2_full_frac1_seed0.pth`. The folder is ignored by git.
17. **Print and return.** The test line prints the best epoch, accuracy, macro-F1, minutes and seconds per epoch. The function returns the row plus `best_epoch`, `val_accuracy`, `seconds_per_epoch` and the `history` dictionary.

What is written to disk: - `results/metrics.csv`: one row changed or added (accuracy and macro-F1 are decimals, for example 0.9827). - `models/mobilenet_v2_full_frac1_seed0.pth`: the best weights only. - The torch cache (the first time only): the ImageNet weights downloaded by torchvision. - Nothing else. The per-epoch history is returned in memory, not written to a file. The best epoch is printed and returned, but not saved in the CSV.

---

6. Why these choices

- **One shared `train()`.** Every model is trained with the same loop, the same optimiser setup, the same split, the same scoring and the same row format. Differences in the results then come from the models, not from the code. This is the brief's main requirement for Part D.
- **A fixed seed.** The seed makes the starting weights, the batch order, the augmentation and the subset repeatable. A teammate can run the same command and get the same numbers (up to tiny differences between machines). The seed is also the way to compare two settings fairly.
- **Several seeds (0, 1, 2).** One run can be lucky or unlucky. With three seeds we can show the mean and the spread. Seeds 0 to 2 were run on Nitin's GPU laptop. Seed 3 is still missing, and seed 4 exists only for three scratch-CNN settings (from Rohan's CPU script).

- **Validation chooses the epoch.** The validation set is a practice exam for decisions. Choosing the best epoch on validation avoids tuning the model to the test set. The test score stays an honest estimate.
- **Test once.** Scoring the test set after training, once per run and never for a decision, means its numbers are not inflated.
- **Macro-F1 as well as accuracy.** The classes have different sizes. Macro-F1 gives each class the same weight, so a weak small class cannot hide behind a good accuracy.
- **Keep the data in memory.** Decoding JPEGs in every epoch was too slow (about 45 to 90 seconds per epoch on a CPU laptop). In memory, the same maths runs in about 4.6 seconds per epoch on the RTX 3050. We checked that the pixels match the original loader.
- **Augmentation on the GPU.** Rotations and flips are exact for satellite tiles. Doing them on the GPU in batches is fast.
- **A log x-axis.** The training sizes grow by about two times at each step (945, 1,890, 4,725, 9,450, 18,900). On a log axis the steps are evenly spaced, and the small sizes are not squeezed together.
- **Mean ± sample SD with `ddof=1`.** `ddof=1` is the correct SD for a small sample of seeds. The error bars show how much the seeds disagree.
- **Replace rows instead of adding them.** A rerun of the same run must not duplicate the row. This keeps the table clean and makes the notebook safe to restart.
- **Only the full-data weights are saved.** The website and the evaluation need full-data models. Saving every small run would fill the disk.
- **Lower learning rate for full fine-tuning.** Pretrained weights are already good, so a big step could damage them. `1e-4` changes them gently.

Points to be careful about in the viva:

- The 100% scratch CNN rows came from Tejash's own notebook helper, not from `train()`. According to the log, `train()` reproduced his mean (92.79% against 92.72%) with seeds 0 to 2.
- Frozen transfer models still update BatchNorm averages (see the list above).
- All sizes get the same budget of up to 20 epochs. Small sets get fewer updates and may be under-trained.
- The study uses only three seeds for the main table. Seed 4 rows exist for the scratch CNN at 5%, 10% and 25% (from Rohan's own CPU script, not `train()`), but they are not in the study table.
- Grad-CAM on 64 x 64 tiles is coarse. The control experiment shows that hiding the hottest and the coldest quarter cost almost the same confidence (0.064 and 0.061), so the heat map is a picture, not proof.

---

7. Numbers you must know

All numbers come from `results/metrics.csv` (48 rows) unless marked. Accuracy in the CSV is a decimal. The table shows percent, mean over seeds 0, 1 and 2, with the sample SD (`ddof=1`). The notebook `05_data_efficiency.ipynb` computes these.

What	Number	Where it comes from
MobileNetV2, fully fine-tuned, all data (18,900 images)	**98.27 ± 0.12** % test accuracy	`metrics.csv`, `model = mobilenet_v2_full`, `fraction = 1.0`, seeds 0, 1, 2 (0.98272, 0.98148, 0.98395). Also in the root README.
Scratch CNN (our own), all data	**92.72 ± 0.45** %	`metrics.csv`, `model = scratch_cnn`, `fraction = 1.0`, seeds 0, 1, 2 (0.93136, 0.92765, 0.92247).
MobileNetV2, 5% (945 images)	**93.69 ± 0.20** %	`metrics.csv`, `mobilenet_v2_full`, `fraction = 0.05`, seeds 0, 1, 2
Scratch CNN, 5% (945 images)	77.07 ± 0.74 %	`metrics.csv`, `scratch_cnn`, `fraction = 0.05`, seeds 0, 1, 2
Gap at 5%	16.62 points	Notebook 05 gap table (MobileNetV2 minus scratch)
Gap at 100%	5.56 points	Notebook 05 gap table
Data-efficiency table, MobileNetV2 (945, 1,890, 4,725, 9,450, 18,900)	93.69, 96.15, 97.26, 97.76, 98.27 %	`metrics.csv`, notebook 05 table

What	Number	Where it comes from
Data-efficiency table, scratch CNN (same sizes)	77.07, 80.98, 84.96, 89.45, 92.72 %	`metrics.csv`, notebook 05 table
Macro-F1, MobileNetV2 all data	98.19 ± 0.13 %	Notebook 05 table (from the macro-F1 column)
Macro-F1, scratch CNN all data	92.48 ± 0.51 %	Notebook 05 table
Speed on RTX 3050	about 4.6 s per epoch	Log (section 2), not printed by the code
Study run time	24 runs in about 20 minutes	Notebook 05 output (sweep time 20.0 minutes)
Total rows in `metrics.csv`	48	Count of the file
Grad-CAM control: mean fall in confidence, hottest 25% hidden	0.064	`results/gradcam_sanity_check.csv` and the script output
Grad-CAM control: coldest 25% hidden	0.061	Same
Grad-CAM control: random 32 x 32 square hidden	0.089	Same
Grad-CAM control: hottest quadrant bottom-right / top-right / bottom-left / top-left	160 / 36 / 4 / 0 of 200	Same file, column `hottest_quadrant`

Notes on the numbers:

- The 100% scratch CNN rows come from Tejash's own notebook helper (Part B). The 100% MobileNetV2 row comes from Rachana's Part C work. They are not re-run here. The notebook uses them as they are.
- Seeds 0, 1 and 2 are used for all tables and for the website's efficiency curves.
- The 5% to 50% rows of the study were run with `train()` on Nitin's RTX 3050. Seed 4 rows (scratch CNN at 5%, 10%, 25%) exist from a CPU script and are not in the three-seed tables.
- Two decimals are rounded. For example, 98.27 comes from 0.982716 in the file.